СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 5
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 6
ВВЕДЕНИЕ 7
1 ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ 9
1.1 Описание предметной области 9
1.2 Анализ существующих аналогов 9
1.2.1 Spotify 9
1.2.2 Apple Music 10
1.2.3 Яндекс Музыка 10
1.3 Техническое задание 11
1.3.1 Определение пользовательских потребностей 11
1.3.3 Определение функциональных потребностей 11
2 ПРОЕКТНАЯ ЧАСТЬ 14
2.1 Описание процесса в нотации IDEF0 14
2.2 Описание процесса в нотации DFD 22
2.3 Описание и обоснование выбора программного обеспечения 24
2.4 Архитектура программной системы 25
3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ 28
3.1 Описание моделей и структур данных 28
3.2 Тестирование программного продукта 33
3.2 Полный листинг 39
ЗАКЛЮЧЕНИЕ 64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 65

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете применяют следующие термины с соответствующими определениями:

Мобильное приложение — программное обеспечение, предназначенное для работы на мобильных устройствах под управлением операционной системы Android.

Плейлист — упорядоченный список музыкальных треков, формируемый пользователем для персонального прослушивания.

API — программный интерфейс приложения, обеспечивающий взаимодействие мобильного клиента с внешними сервисами.

Локальное хранилище — механизм хранения данных на устройстве пользователя для обеспечения быстрого доступа и офлайн-работы приложения.

Пользовательский интерфейс — совокупность экранов, элементов управления и визуальных компонентов, обеспечивающих взаимодействие пользователя с приложением.

Метаданные трека — структурированная информация о музыкальном треке (название, исполнитель, альбом, жанр, продолжительность).

Архитектура приложения — логическая структура программной системы, определяющая взаимодействие её компонентов.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете применяют следующие сокращения и обозначения:

API – Application Programming Interface

DFD (Data flow diagrams) — диаграмма потоков данных

IDEF (Integrated Definition for Function Modeling) — комплексное описание

SDK – Software Development Kit

UI – User Interface

XML – eXtensible Markup Language

UX — User Experience

SDK — Software Development Kit

JSON — JavaScript Object Notation

DB — Database (база данных)

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка мобильного приложения «Playlist Maker», предназначенное для поиска музыкальных треков, создания персональных плейлистов и управления пользовательской музыкальной коллекцией на устройствах под управлением операционной системы Android. Приложение ориентировано на пользователей, которые хотят удобно находить музыку, структурировать её по собственным предпочтениям и иметь быстрый доступ к сохранённым плейлистам.

Актуальность темы обусловлена стремительным ростом потребления цифрового аудиоконтента и популярностью стриминговых сервисов. Современные пользователи ожидают от мобильных приложений высокой скорости работы, простоты интерфейса и возможности персонализации контента.

Проект «Playlist Maker» представляет собой прототип мобильного приложения, в котором реализованы базовые функции музыкального сервиса: поиск треков через внешний API, отображение результатов поиска, сохранение понравившихся композиций и формирование пользовательских плейлистов. Для повышения удобства использования часть данных хранится локально на устройстве пользователя, что позволяет обращаться к сохранённой информации без постоянного подключения к сети Интернет.

Объектом исследования в данной курсовой работе является мобильное приложение для работы с музыкальным контентом. Предметом исследования являются методы проектирования и разработки клиентской части мобильных приложений, а также способы организации хранения и обработки данных о музыкальных треках и плейлистах.

В исследовательской части курсовой работы проводится изучение существующих программных решений и подходов, выполняется анализ выбранной предметной области, а также разрабатываются диаграммы в нотациях

IDEF0 и DFD, отражающие ключевые процессы взаимодействия пользователей с системой, хранилищами данных и пользовательским интерфейсом. В проектной части формируются функциональные требования к разрабатываемой системе, обосновывается выбор архитектурных решений и используемого технологического стека. В технологической части описывается реализация основных компонентов приложения, включая элементы пользовательского интерфейса и обработку данных, а также приводятся результаты тестирования функций программного продукта.

1 ИССЛЕДОВАТЕЛЬСКАЯ ЧАСТЬ

1.1 Описание предметной области

Предметная область курсовой работы связана с разработкой мобильного приложения для поиска и управления музыкальными треками и плейлистами. В условиях широкого распространения цифровых музыкальных сервисов особую актуальность приобретают задачи удобного доступа к музыкальному контенту и его персонализации под потребности пользователя.

В рамках данной предметной области пользователь с помощью мобильного приложения осуществляет поиск музыкальных треков, просматривает информацию о них и формирует собственные плейлисты. Источником данных о музыкальных композициях являются внешние сервисы, предоставляющие информацию через программные интерфейсы. Для хранения выбранных пользователем треков и плейлистов используется локальное хранилище данных на мобильном устройстве.

Таким образом, предметная область охватывает процессы поиска музыкального контента, его обработки и хранения, а также организацию взаимодействия пользователя с системой через мобильный интерфейс.

1.2 Анализ существующих аналогов

При разработке мобильного приложения необходим анализ существующих рыночных решений. Это позволяет, изучив сильные и слабые стороны конкурентов, создать продукт, способный составить им конкуренцию. В качестве примера рассмотрим популярные музыкальные сервисы, которые позволяют пользователям искать, воспроизводить и управлять треками и плейлистами.

1.2.1 Spotify

Spotify - один из крупнейших мировых стриминговых музыкальных сервисов, предоставляющий доступ к обширному каталогу музыкальных композиций различных жанров и исполнителей. Приложение поддерживает

поиск треков, создание плейлистов, рекомендации на основе предпочтений пользователя и синхронизацию между устройствами.

К достоинствам Spotify относятся удобный и интуитивно понятный пользовательский интерфейс, развитая система персональных рекомендаций, а также большое количество доступного музыкального контента. Основными недостатками сервиса являются необходимость регистрации и подписки для доступа ко всем функциям, а также ограниченные возможности использования без подключения к сети Интернет.

1.2.2 Apple Music

Apple Music представляет собой музыкальный стриминговый сервис, ориентированный на экосистему устройств компании Apple, но также доступный и на платформе Android. Приложение предоставляет функции поиска музыкальных треков, создания и редактирования плейлистов, а также прослушивания музыкальных подборок.

К достоинствам Apple Music можно отнести высокое качество аудиоконтента, глубокую интеграцию с устройствами и сервисами Apple, а также удобную организацию музыкальной библиотеки. К недостаткам относятся обязательная платная подписка, отсутствие бесплатного режима использования и ограниченная гибкость настройки интерфейса.

1.2.3 Яндекс Музыка

Яндекс Музыка — популярный музыкальный сервис, ориентированный преимущественно на русскоязычную аудиторию. Приложение предоставляет возможности поиска треков, создания плейлистов, а также персональных рекомендаций на основе истории прослушиваний.

К достоинствам Яндекс Музыки относятся удобный интерфейс, качественная система рекомендаций и хорошая адаптация под локальный музыкальный рынок. Среди недостатков можно выделить необходимость авторизации, наличие платной подписки для полного доступа к функционалу, зависимость от интернет-соединения, а также отсутствие множества треков иностранных исполнителей.

Для наглядного сравнения рассмотренных аналогов приведена сводная таблица (см. табл. 1).

Таблица 1 – Таблица сравнения существующих аналогов

Характеристика	Spotify	Apple Music	Яндекс Музыка
Поиск музыкальных треков	Да	Да	Да
Создание плейлистов	Да	Да	Да
Персональные рекомендации	Да	Да	Да
Бесплатный доступ	Ограниченный	Нет	Ограниченный
Необходимость регистрации	Да	Да	Да
Работа без интернета	Частично	Частично	Частично

1.3 Техническое задание

1.3.1 Определение пользовательских потребностей

На основе анализа аналогичных музыкальных приложений, можно выделить ключевые ожидания пользователей от мобильного приложения «Playlist Maker»:

- простота и интуитивность интерфейса;
- возможность быстрого поиска нужных треков;
- минимум ограничений без регистрации;
- отсутствие навязчивой рекламы;
- возможность сохранять понравившиеся композиции;
- возможность создания собственных плейлистов.

Таким образом, пользователи ожидают от создаваемого приложения:

- удобный интерфейс для поиска и управления музыкой;
- доступ к информации без обязательной регистрации;
- сохранение избранных треков;
- возможность создавать и редактировать плейлисты;
- стабильную работу приложения при медленном соединении и корректную обработку ошибок.

1.3.3 Определение функциональных потребностей

На основе анализа предметной области и пользовательских ожиданий сформулированы следующие функциональные требования к мобильному

приложению Playlist Maker:

1. Поиск музыки:
    - интеграция с iTunes Search API для поиска треков по названию, исполнителю или альбому;
    - реализация истории поиска с ограничением до 10 запросов;
    - устранение дубликатов в истории поиска;
    - автоматическое скрытие клавиатуры при взаимодействии с элементами интерфейса;
    - обработка ошибок сети с возможностью повторного запроса.
2. Управление плейлистами:
    - создание плейлистов с возможностью задания названия, описания и обложки;
    - добавление и удаление треков из плейлистов;
    - удаление плейлистов через долгое нажатие или иконку удаления;
    - хранение плейлистов в локальной базе данных Room с сохранением данных между сеансами.
3. Управление избранным:
    - добавление и удаление треков в избранное;
    - отображение списка избранных треков на отдельном экране;
    - удаление треков из избранного через долгое нажатие.
4. Навигация и интерфейс:
    - реализация навигации между основными экранами: Поиск, Плейлисты, Избранное, Настройки;
    - использование Jetpack Navigation для управления переходами;
    - поддержка динамического переключения между светлой и тёмной темами;
    - интеграция библиотеки Coil для асинхронной загрузки обложек треков.
5. Локальное хранение данных:

- хранение плейлистов, треков и истории поиска в локальной базе данных Room;
- использование DataStore для сохранения настроек темы и истории поиска;
- обеспечение работы приложения без постоянного подключения к сети.

6. Дополнительные функции:
    - шеринг приложения через стандартные средства Android;
    - отправка email в поддержку;
    - открытие пользовательского соглашения.
7. Производительность и UX:
    - оптимизация интерфейса для работы с клавиатурой;
    - использование ViewModel для управления состоянием экранов;
    - отображение placeholder-изображений при загрузке обложек;
    - корректная работа BottomSheet для выбора плейлистов.

2 ПРОЕКТНАЯ ЧАСТЬ

2.1 Описание процесса в нотации IDEF0

IDEF0 — методология функционального моделирования и графическая нотация для описания бизнес-процессов, проектирования систем и их взаимодействий. Она представляет систему в виде иерархии диаграмм. Каждая функция представляется в виде прямоугольника, а взаимодействие с внешней средой — с помощью стрелок четырёх типов:

- вход (Input) — данные или ресурсы, преобразуемые функцией;
- выход (Output) — результаты выполнения функции;
- управление (Control) — правила, стандарты и ограничения, определяющие поведение функции;
- механизм (Mechanism) — инструменты, технологии и ресурсы, обеспечивающие выполнение функции.

В данном случае рассматривается контекстная диаграмма системы Playlist Maker, которая описывает весь процесс работы мобильного приложения.

Система имеет следующие входы:

- действия пользователя (посик треков, создание плейлистов, добавление трека в избранное/плейлист, выбор темы приложения и другие);
- результаты поиска из iTunes Search API;
- сохраненные плейлисты и избранные треки;
- история поисковых запросов.

Управляющие воздействия (Control) включают:

- бизнес-логика приложения (правила поиска, управления плейлистами и избранным);
- требования к хранению данных;

- шаблоны интерфейса, задающие визуальное отображение элементов приложения;
- настройки пользователя (тема интерфейса)

Механизмы выполнения функций:

- локальная база данных (Room) для хранения пользовательских плейлистов и избранных треков;
- iTunes Search API для получения информации о треках;
- Android SDK (набор системных библиотек и базовых функций платформы Android) и компоненты UI;
- ресурсы из папки res/drawable (иконки и изображения обложек треков).

На выходе система формирует:

- интерфейс мобильного приложения, который предоставляет пользователю к каталогу треков, плейлистов, избранному и другим функциям.

Данная диаграмма процесса использования приложения в нотации IDEF0 представлена на рисунке 1.

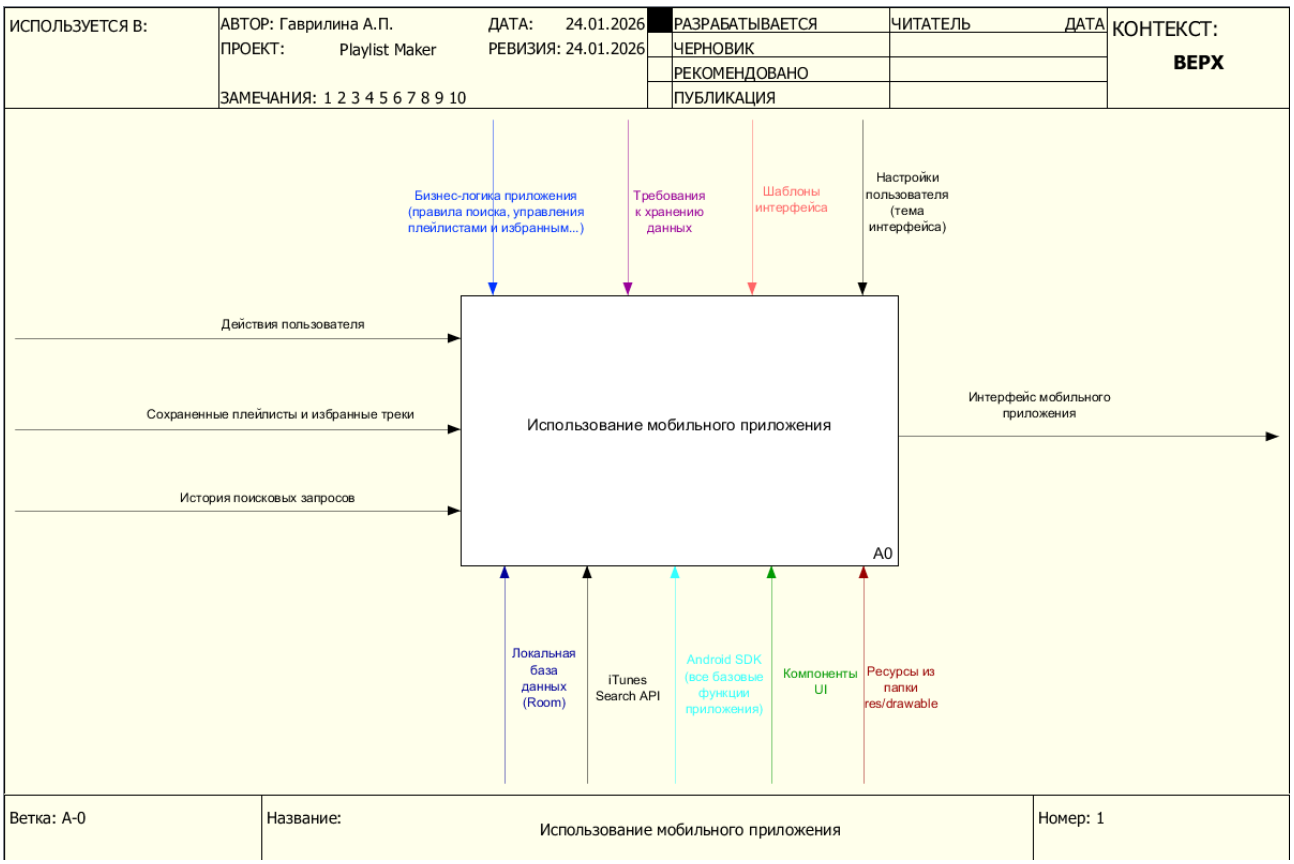

Рисунок 1 - Описание процесса в нотации IDEF0

Процесс использования мобильного приложения включает в себя поиск треков, управление избранным, управление плейлистами, просмотр деталей трека и настройку приложения. Декомпозиция процесса использования приложения представлена на рисунке 2.

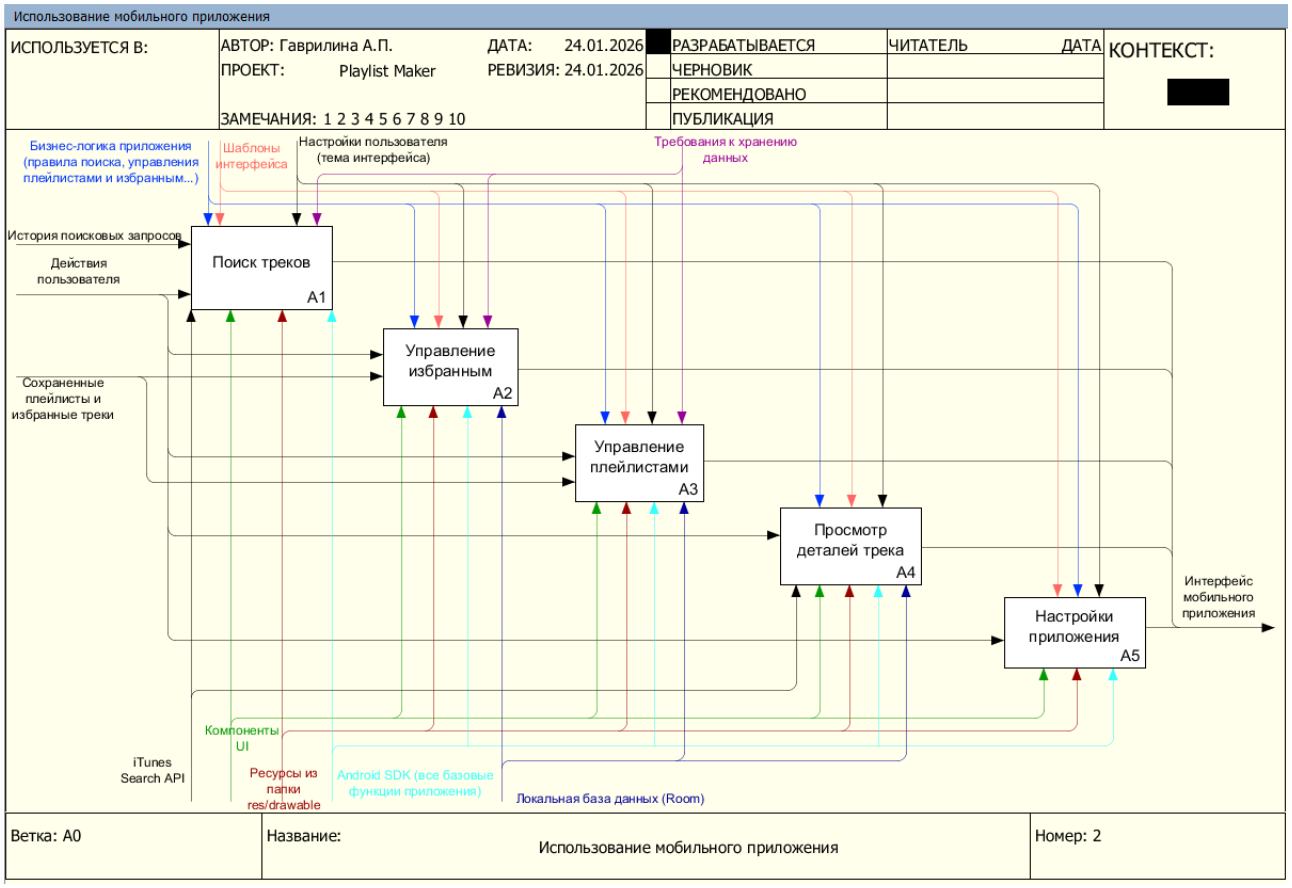

Рисунок 2 - Декомпозиция процесса в нотации IDEF0

Подпроцесс поиска треков обеспечивает поиск музыкальных треков по запросу пользователя (название, исполнитель, альбом), получение результатов из внешнего сервиса и отображение найденных данных, а также формирование и показ истории поисковых запросов в интерфейсе приложения (см. рис. 3).

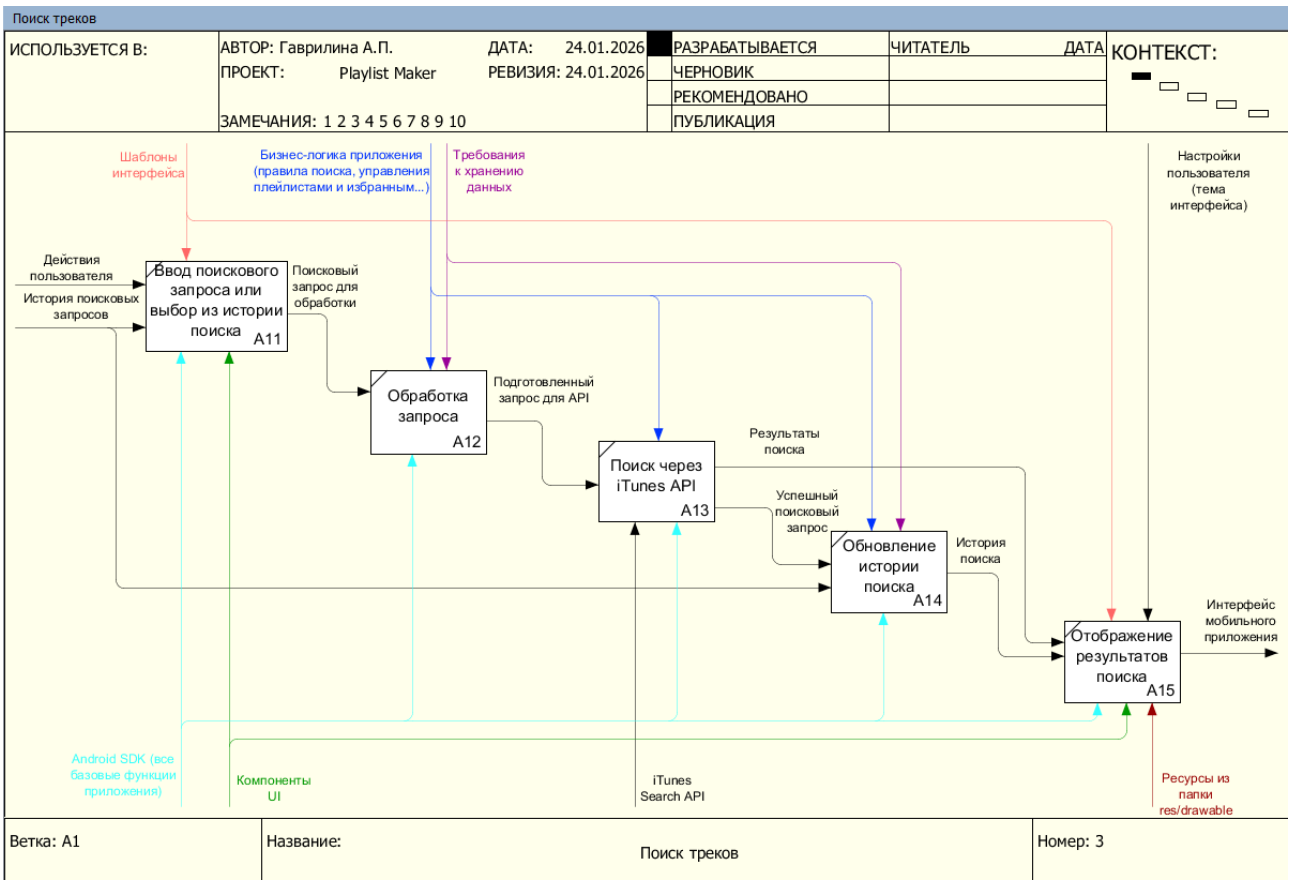

Рисунок 3 - Декомпозиция процесса поиска треков в нотации IDEF0

Подпроцесс управления избранным предназначен для добавления и удаления треков в список избранного, а также для отображения актуального списка избранных треков в интерфейсе мобильного приложения (см. рис. 4).

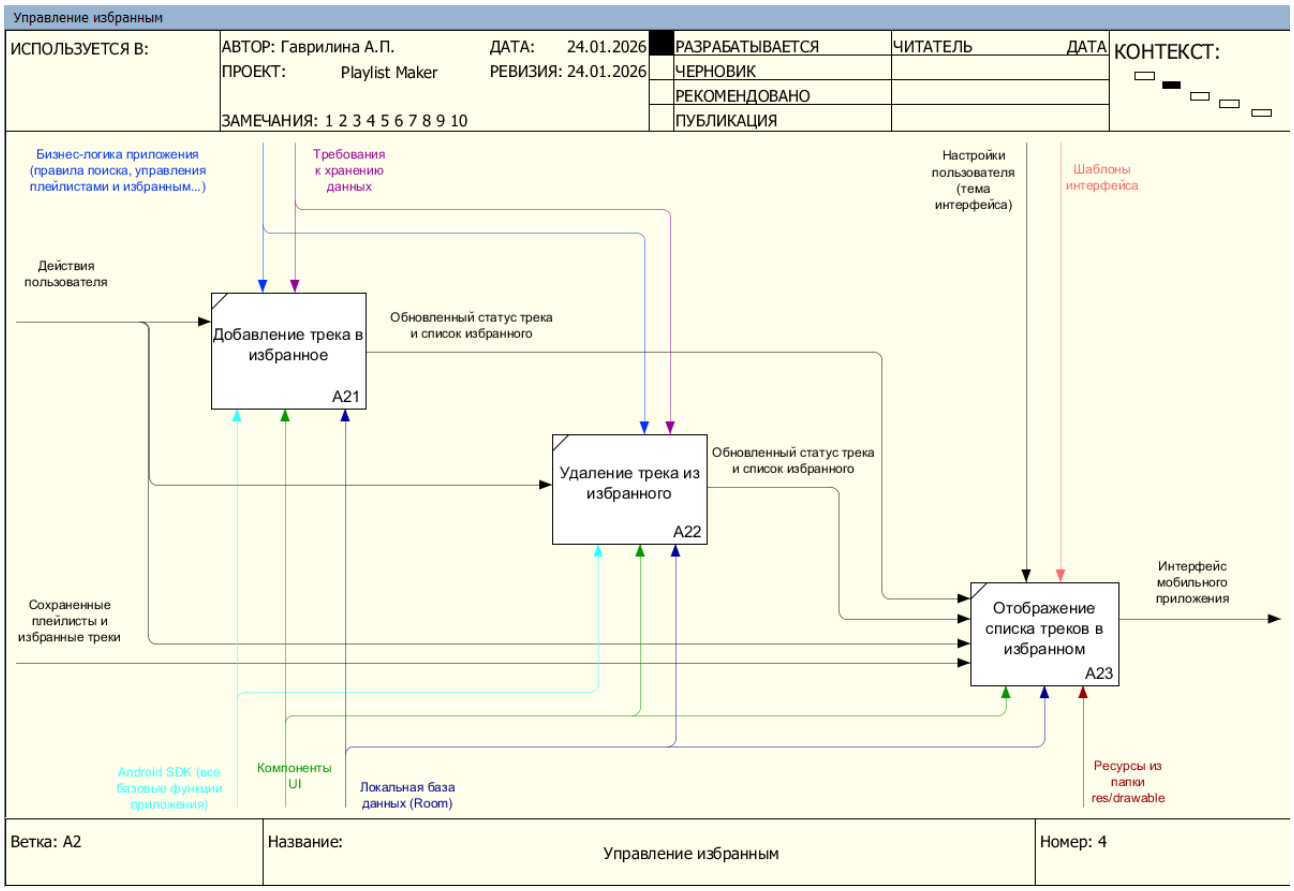

Рисунок 4 - Декомпозиция процесса управления избранным в нотации IDEF0

Подпроцесс управления плейлистами обеспечивает создание и удаление пользовательских плейлистов, управление их содержимым, а также отображение списка плейлистов и деталей выбранного плейлиста в интерфейсе приложения (см. рис. 5).

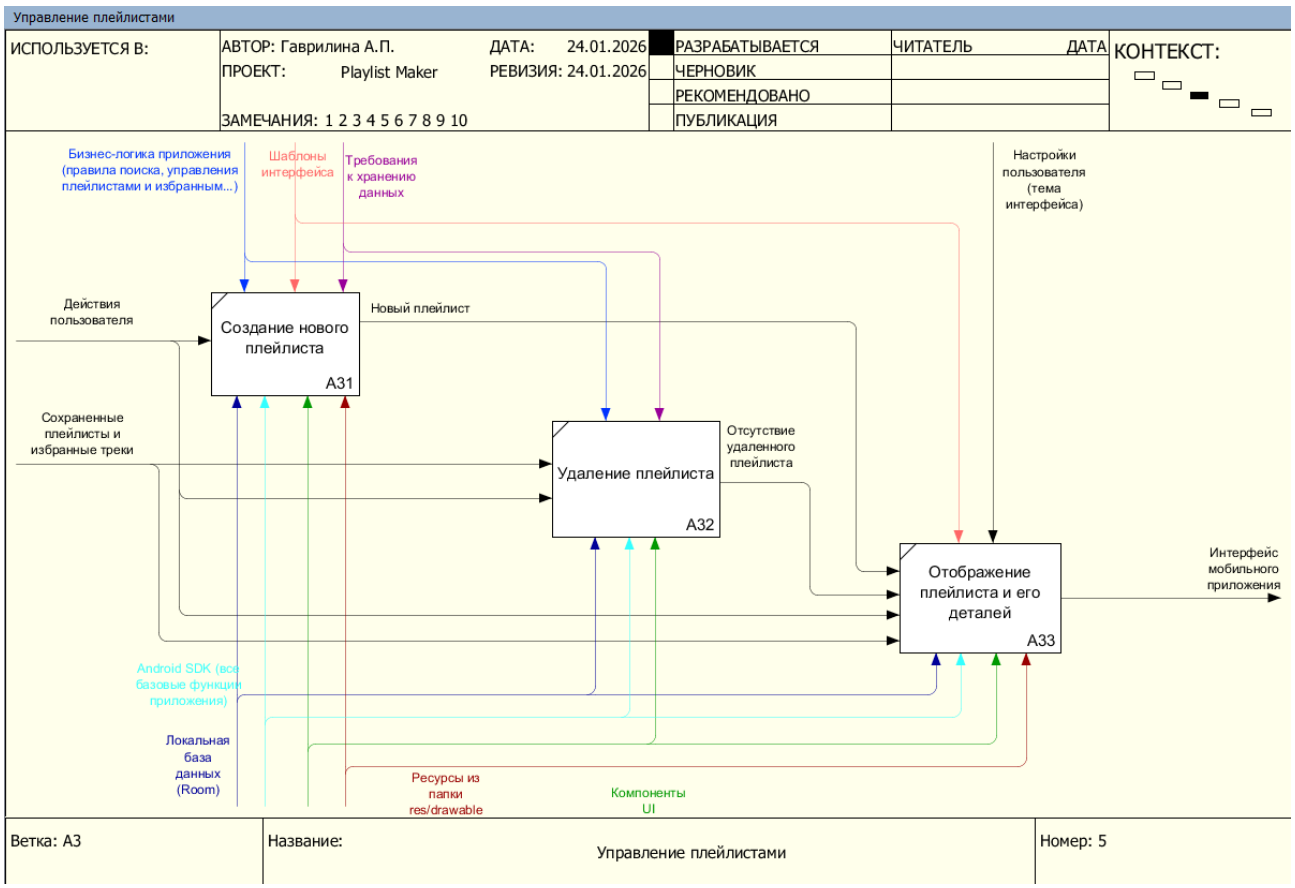

Рисунок 5 - Декомпозиция процесса управления плейлистами в нотации IDEF0

Подпроцесс просмотра деталей трека отвечает за отображение детальной информации о выбранном треке и предоставляет пользователю возможность выполнить действия с треком, такие как добавление в избранное или в плейлист (см. рис. 6).

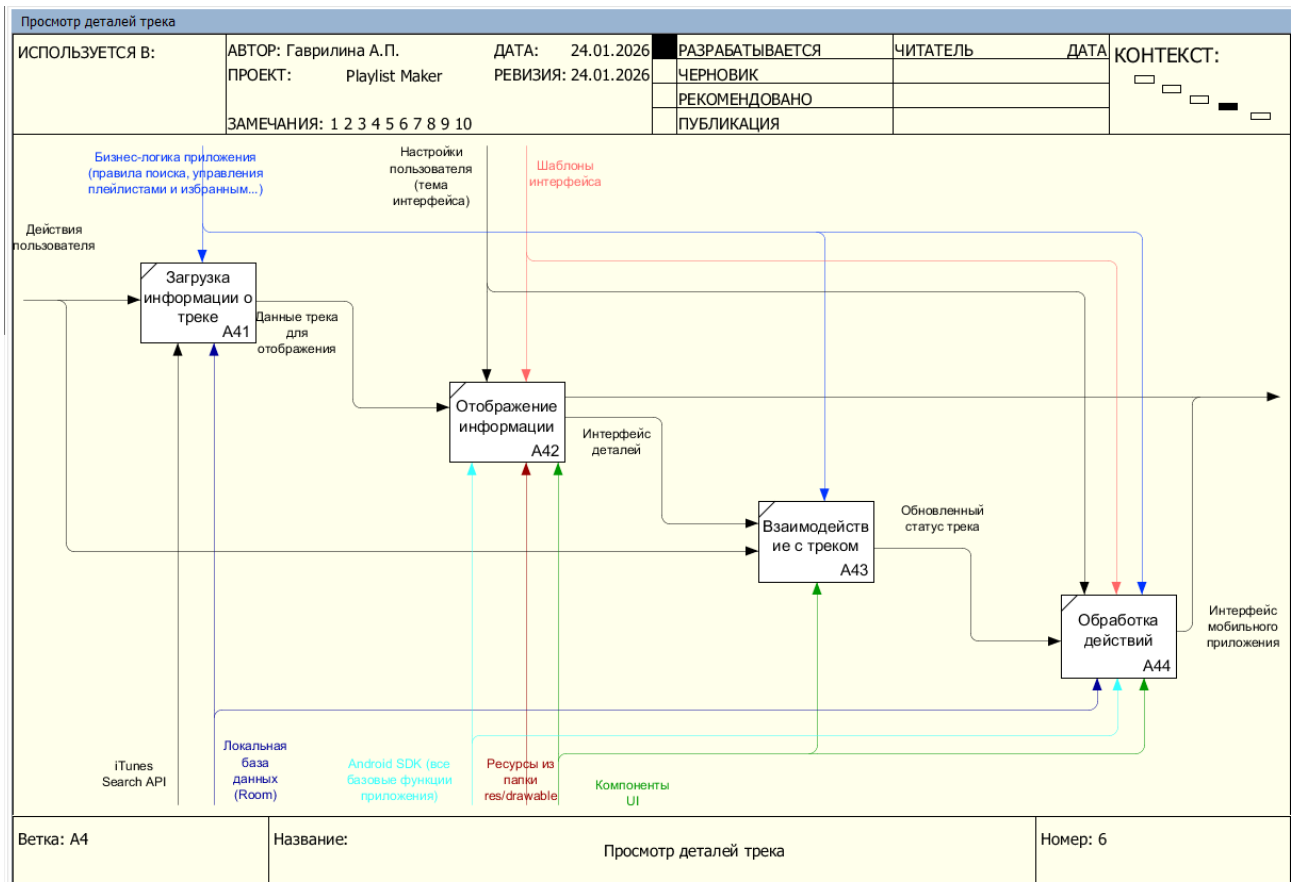

Рисунок 6 - Декомпозиция процесса просмотр деталей трека в нотации IDEF0

Подпроцесс настройки приложения обеспечивает изменение пользовательских настроек приложения, включая выбор темы интерфейса, доступ к функциям шеринга, связь с разработчиком и просмотр пользовательского соглашения (см. рис. 7).

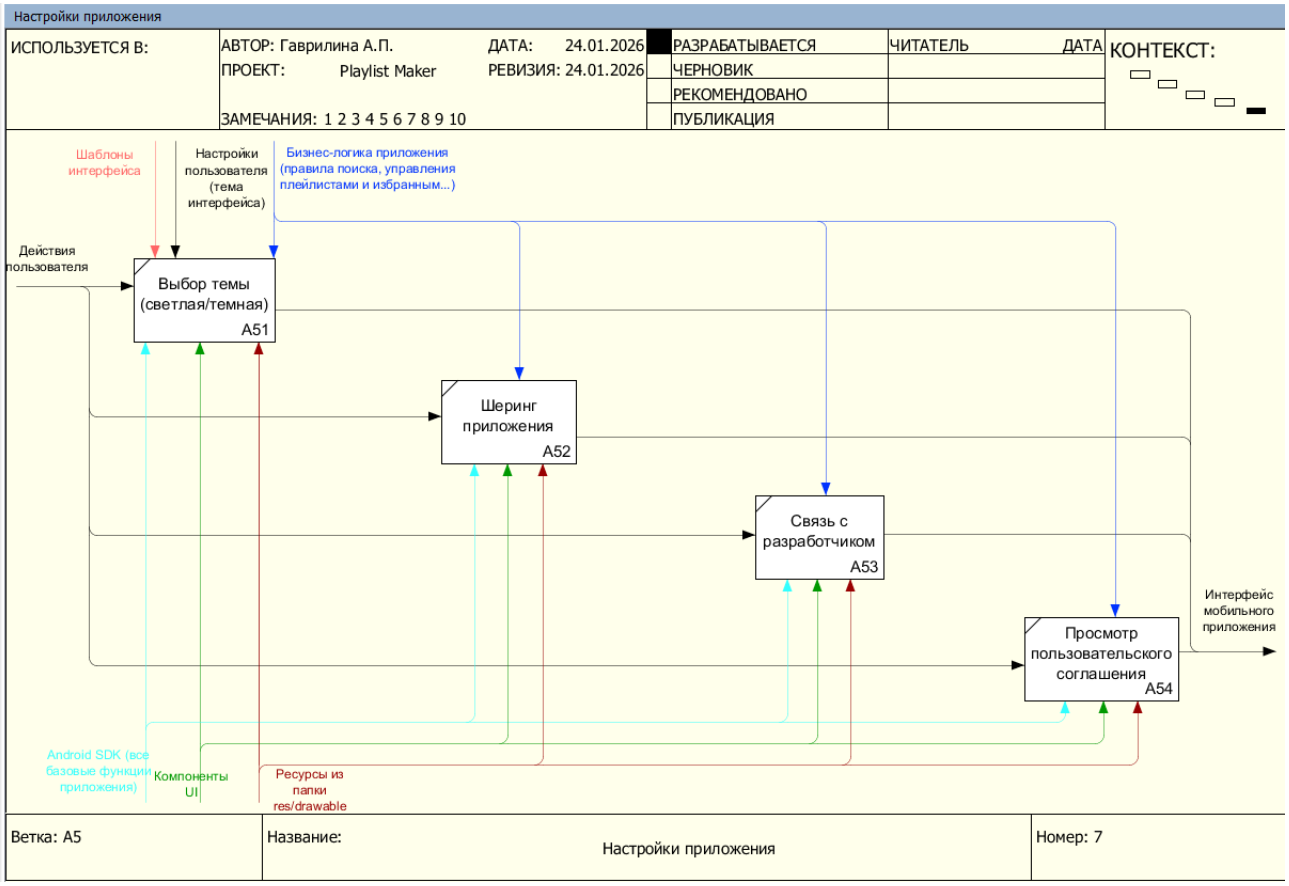

Рисунок 7 - Декомпозиция процесса настройки приложения в нотации IDEF0

2.2 Описание процесса в нотации DFD

Диаграмма в нотации DFD отображает потоки данных в системе (процессе). В нотации Гейна - Сарсона внешние сущности (источники или получатели данных) отображаются прямоугольниками, процессы (действия, преобразующие данные) – прямоугольниками с закругленными краями, хранилища – прямоугольниками без правой стороны, а потоки данных – стрелками.

DFD диаграмма, отображающая потоки данных мобильного приложения Playlist maker, представлена на рисунке 8. Данная диаграмма потока данных (DFD) описывает архитектуру мобильного приложения Playlist Maker, визуализируя: 5 основных бизнес-процессов приложения, 4 хранилища данных, 2 внешние сущности (пользователь и iTunes API) и сами потоки данных, связывающие компоненты системы.

Для поиска треков на вход поступает от БД, хранящей историю поиска, 10 последних успешных запросов пользователя. Пользователь их получает и отправляет поисковый запрос созданный самостоятельно или выбранные из списка истории поиска. Поисковый запрос передается iTunes Search API, который возвращает результаты поиска, которые в свою очередь отправляются пользователю, а также записываются в базы данных, которые хранят историю поиска и информацию о треках.

При добавлении или удалении пользователем трека из избранного в локальную базу данных треков передается измененный статус трека. В обратную сторону база данные присылает список избранных треков, которые передаются пользователю.

При работе с плейлистами происходит действия подобные тем, которые происходят при управлении избранным, помимо работы с локальной базой плейлистов.

При просмотре деталей трека приложение получает ID выбранного трека, по которому могут вносится изменения в данные о треке (плейлисты, избранное) в базу данных треков. От самой базы данных система получает информацию о треке.Также система получает информацию о плейлистах из базы данных плейлистов, чтобы предоставить пользователю возможность добавить трек в выбранные плейлист.

При работе с настройками приложения от пользователя система получает какую-либо команду связанную с одноименным экраном. Система меняет статус темы на противоположный, отправляя информацию об этом в базу данных настройки темы и получает обратно статус текущей темы.

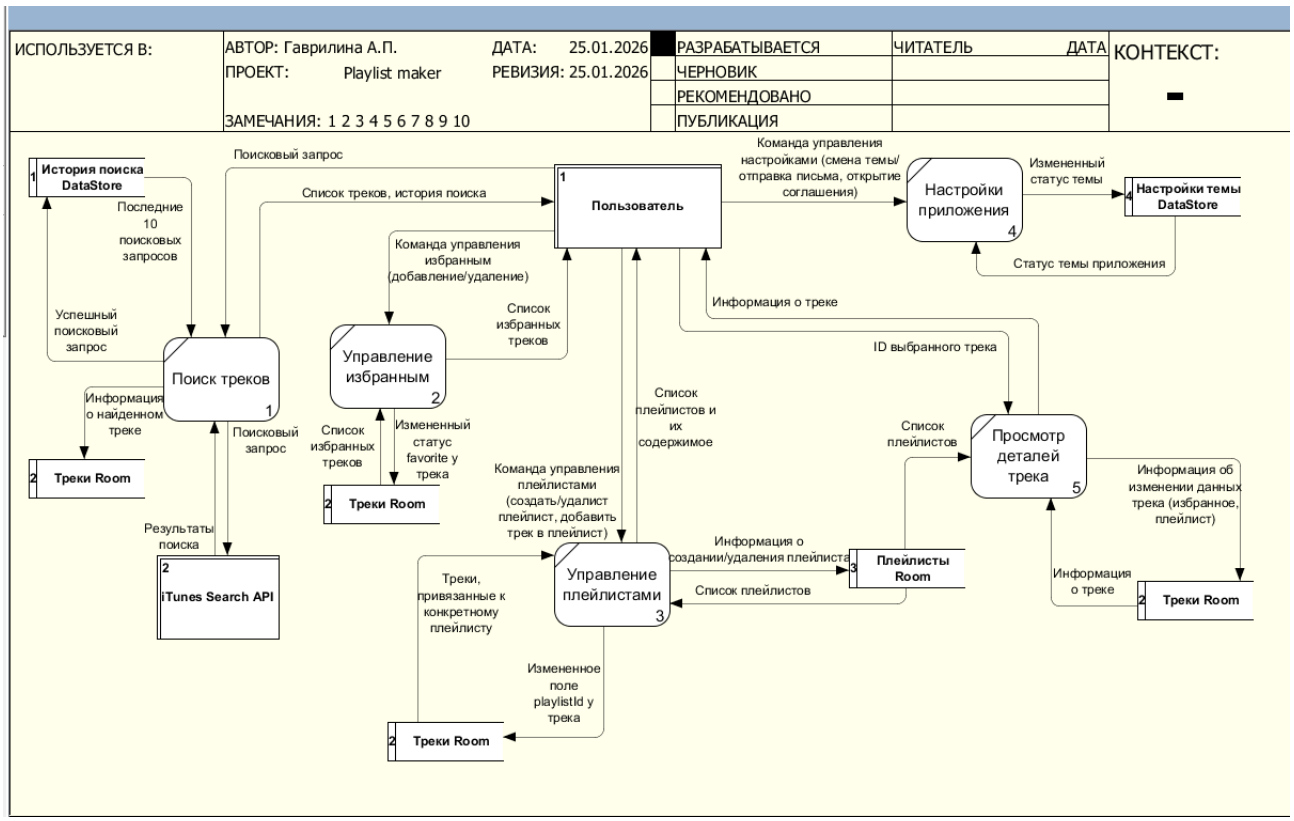

Рисунок 8 - Декомпозиция процесса использования приложения в нотации DFD

2.3 Описание и обоснование выбора программного обеспечения

Для разработки мобильного приложения Playlist Maker была выбрана среда разработки Android Studio, являющаяся официальным инструментом для создания приложений под платформу Android. Она предоставляет необходимые средства для проектирования интерфейса, написания кода, тестирования и отладки приложения.

В качестве языка программирования используется Kotlin, так как он официально поддерживается платформой Android, обладает лаконичным синтаксисом и снижает вероятность ошибок при работе с данными. Использование Kotlin упрощает разработку и повышает надежность приложения.

Для получения информации о музыкальных треках применяется iTunes Search API, предоставляющий данные о композициях, исполнителях и альбомах

в формате JSON. Выбор данного API обусловлен его открытостью, простотой использования и отсутствием необходимости авторизации.

Для выполнения сетевых запросов и обработки ответов используется библиотека Retrofit в сочетании с Gson, что обеспечивает удобную работу с REST API и автоматическое преобразование данных.

Хранение пользовательских плейлистов и избранных треков осуществляется локально с использованием SharedPreferences, что является достаточным решением для прототипа приложения.

2.4 Архитектура программной системы

При разработке приложения структура Kotlin-классов была логически организована по пакетам, каждый из которых отвечает за отдельную область функциональности:

1. creator содержит классы для создания основных компонентов приложения:
    - Creator — отвечает за инициализацию и настройку зависимостей.
2. data содержит слои для работы с данными:
    - converter (преобразователи между моделями, сущностями и DTO): PlaylistConverter.kt, TrackConverter.kt — конвертация между Playlist, Track, PlaylistEntity, TrackEntity и TrackDto.
    - database (работа с локальной базой данных Room):

      dao (интерфейсы доступа к данным): PlaylistsDao, TracksDao — запросы к таблицам плейлистов и треков.

      entity (сущности базы данных): PlaylistEntity, TrackEntity, PlaylistTrackCrossRef — таблицы и связи.
    - db (конфигурация базы данных и менеджеры): AppDatabase — основная конфигурация БД, ImageFileManager — управление файлами изображений.
    - dto (объекты передачи данных для сетевых запросов и ответов): BaseResponse, TrackDto, TracksSearchRequest, TracksSearchResponse.

- network (сетевое взаимодействие): RetrofitNetworkClient — реализация сетевого клиента, TracksRepositoryImpl — реализация репозитория для работы с треками.
- preferences (управление настройками приложения): SearchHistoryPreferences, ThemePreferences — хранение истории поиска и темы;
- PlaylistsRepositoryImpl, SearchHistoryRepositoryImpl — реализации репозиториев для плейлистов и истории поиска.

3. domain содержит бизнес-логику и интерфейсы:
    - models (основные модели приложения): Playlist, Track, Word — доменные модели;
    - iTunesApiService, NetworkClient, PlaylistsRepository, SearchHistoryRepository, TracksRepository — интерфейсы для API и репозиториев.
4. ui - пользовательский интерфейс:
    - activity: AppActivity — точка входа в приложение.
    - favorites (экран избранного): FavoritesScreen.kt — отображение списка избранных треков.
    - main (главный экран): MainScreen — основной UI с поиском и навигацией.
    - navigation (навигация между экранами): Navigation, PlaylistHost — управление переходами и хост плейлистов.
    - playlists (управление плейлистами): CreatePlaylistScreen, CreatePlaylistViewModel — экран и ViewModel для создания нового плейлиста, PlaylistScreen, PlaylistDetailViewModel — экран детального просмотра плейлиста и ViewModel для управления его содержимым, PlaylistsScreen, PlaylistViewModel — экран со списком всех плейлистов и ViewModel для его управления.

- search (поиск треков): SearchScreen, SearchState, SearchViewModel — экран поиска, класс состояния экрана поиска и ViewModel для экрана поиска.
- settings (настройки): SettingsScreen — экран настроек приложения.
- theme (тема приложения): Color, Type — цветовая палитра и типографика темы, Theme — объект, настраивающий MaterialTheme на основе выбранной темы, ThemeViewModel — ViewModel, который управляет состоянием темы, сохраняет и загружает его через ThemePreferences.
- trackdetails (детали трека): TrackDetailsScreen, TrackDetailsViewModel — экран с подробной информацией о выбранном треке и его ViewModel.

Также используемые изображения и иконки хранятся в res/drawable, а строковые значения strings.xml и заданные расстояния dimens.xml в папке res/values

Данная архитектура следует принципам чистой архитектуры и разделения ответственности, что обеспечивает модульность, тестируемость и легкость поддержки кода.

3 ТЕХНОЛОГИЧЕСКАЯ ЧАСТЬ

3.1 Описание моделей и структур данных

В пакете domain/model размещены основные сущности предметной области приложения Playlist Maker.

Класс Playlist представляет пользовательский плейлист и используется для группировки треков, управления их составом и отображения информации о плейлисте (см. рис.9).

```
package com.practicum.myapplication.domain.models

16 Usages
data class Playlist(
    val id: Long = 0,
    val name: String,
    val description: String,
    val coverImageUri: String? = null,  // URI обложки плейлиста
    var tracks: List<Track> = emptyList()
)
```

Рисунок 9 – Data-класс Playlist

Класс Track описывает музыкальный трек как логический объект и содержит его основные характеристики (см. рис. 10).

```
package com.practicum.myapplication.domain.models

data class Track(
    val id: Long,
    val trackName: String,
    val artistName: String,
    val trackTime: String,
    val image: String,
    var favorite: Boolean,
    var playlistId: List<Long> = emptyList()
)
```

Рисунок 10 – Data-класс Track

Класс Word используется для представления отдельных элементов поискового запроса и участвует в бизнес-логике обработки поиска музыкальных композиций (см. рис. 11).

```
package com.practicum.myapplication.domain.models

data class Word(
    val word: String,
    var count: Int = 1
)
```

Рисунок 11 – Data-класс Word

TrackEntity является сущностью базы данных, предназначенной для хранения информации о музыкальных треках. Класс содержит основные характеристики трека, такие как идентификатор, название, исполнитель, длительность и ссылка на обложку. Данная сущность используется для сохранения треков, добавленных пользователем в избранное или в плейлисты, а также для обеспечения доступа к данным без постоянного подключения к сети Интернет (см. рис. 12).

```
TrackEntity.kt
1   package com.practicum.myapplication.data.database.entity
2
3   import androidx.room.Entity
4   import androidx.room.PrimaryKey
5
    45 Usages
6   @Entity(tableName = "tracks")
7   data class TrackEntity(
8       @PrimaryKey
9       val id: Long,
10      val trackName: String,
11      val artistName: String,
12      val trackTime: String,
13      val image: String,
14      val favorite: Boolean = false
15  )
```

Рисунок 12 – Data-класс TrackEntity

PlaylistEntity представляет собой сущность локальной базы данных, описывающую пользовательский плейлист. Данный класс используется для хранения основной информации о плейлисте, включая его идентификатор, название, описание и путь к обложке. Сущность применяется при создании, редактировании и отображении плейлистов, а также при их сохранении в локальном хранилище данных (см. рис. 13).

PlaylistEntity.kt

```
1   package com.practicum.myapplication.data.database.entity
2
3   import androidx.room.Entity
4   import androidx.room.PrimaryKey
5
    23 Usages
6   @Entity(tableName = "playlists")
7   data class PlaylistEntity(
8       @PrimaryKey(autoGenerate = true)
9       val id: Long = 0,
10      val name: String,
11      val description: String,
12      val coverImageUri: String? = null
13  )
```

Рисунок 13 – Data-класс PlaylistEntity

PlaylistTrackCrossRef используется для реализации связи типа «многие-ко-многим» между плейлистами и треками в локальной базе данных. Сущность содержит идентификаторы плейлиста и трека и позволяет одному плейлисту включать несколько треков, а одному треку принадлежать нескольким плейлистам. Использование данной структуры обеспечивает гибкость при управлении составом пользовательских плейлистов (см. рис. 14).

```kotlin
PlaylistTrackCrossRef.kt
package com.practicum.myapplication.data.database.entity

import androidx.room.Entity

11 Usages
@Entity(
    tableName = "playlist_track_cross_ref",
    primaryKeys = ["playlistId", "trackId"]
)
data class PlaylistTrackCrossRef(
    val playlistId: Long,
    val trackId: Long
)
```

Рисунок 14 – Data-класс PlaylistTrackCrossRef

PlaylistsDao представляет собой интерфейс доступа к данным (DAO), предназначенный для работы с сущностями плейлистов в базе данных. Он содержит методы для добавления, удаления, обновления и получения плейлистов, а также для загрузки плейлистов вместе с входящими в них треками. Данный интерфейс обеспечивает абстракцию доступа к данным и упрощает взаимодействие бизнес-логики приложения с локальным хранилищем (см. рис. 15).

```
package com.practicum.myapplication.data.database.dao

import ...

11 Usages  1 Implementation
@Dao
interface PlaylistsDao {
    1 Usage  1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertPlaylist(playlist: PlaylistEntity): Long

    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM playlists")
    fun getAllPlaylists(): Flow<List<PlaylistEntity>>

    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM playlists WHERE id = :playlistId")
    fun getPlaylistById(playlistId: Long): Flow<PlaylistEntity?>

    1 Usage  1 Implementation
    @Query( value = "DELETE FROM playlists WHERE id = :playlistId")
    suspend fun deletePlaylistById(playlistId: Long)

    // Методы для связи треков и плейлистов
    1 Usage  1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCrossRef(crossRef: PlaylistTrackCrossRef)

    1 Usage  1 Implementation
    @Query( value = "DELETE FROM playlist_track_cross_ref WHERE playlistId = :playlistId AND trackId = :trackId")
    suspend fun deleteCrossRef(playlistId: Long, trackId: Long)

    2 Usages  1 Implementation
    @Query( value = "DELETE FROM playlist_track_cross_ref WHERE playlistId = :playlistId")
    suspend fun deleteAllCrossRefsForPlaylist(playlistId: Long)

    2 Usages  1 Implementation
    @Query( value = "SELECT trackId FROM playlist_track_cross_ref WHERE playlistId = :playlistId")
    suspend fun getTrackIdsForPlaylist(playlistId: Long): List<Long>

    6 Usages  1 Implementation
    @Query( value = "SELECT playlistId FROM playlist_track_cross_ref WHERE trackId = :trackId")
    suspend fun getPlaylistIdsForTrack(trackId: Long): List<Long>
}
```

Рисунок 15 – Интерфейс PlaylistsDao

TrackDao — интерфейс доступа к данным, используемый для работы с сущностью TrackEntity. Он предоставляет методы для сохранения треков, получения списка сохранённых композиций, а также удаления и обновления данных. Интерфейс применяется при реализации функциональности избранных треков и при добавлении треков в пользовательские плейлисты (см. рис. 16).

```
package com.practicum.myapplication.data.database.dao

import ...

11 Usages   1 Implementation
@Dao
interface TracksDao {
    3 Usages   1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertTrack(track: TrackEntity)

    1 Usage   1 Implementation
    @Query( value = "SELECT * FROM tracks WHERE trackName = :trackName AND artistName = :artistName")
    fun getTrackByNameAndArtist(trackName: String, artistName: String): Flow<TrackEntity?>

    1 Usage   1 Implementation
    @Query( value = "SELECT * FROM tracks WHERE favorite = 1")
    fun getFavoriteTracks(): Flow<List<TrackEntity>>

    1 Usage   1 Implementation
    @Query( value = "SELECT * FROM tracks WHERE id = :id")
    fun getTrackById(id: Long): Flow<TrackEntity?>

    1 Implementation
    @Update
    suspend fun updateTrack(track: TrackEntity)

    1 Implementation
    @Query( value = "SELECT * FROM tracks")
    suspend fun getAllTracks(): List<TrackEntity>

    2 Usages   1 Implementation
    @Query( value = "SELECT * FROM tracks")
    fun getAllTracksFlow(): Flow<List<TrackEntity>>

    1 Usage   1 Implementation
    @Query( value = "SELECT * FROM tracks WHERE id = :id")
    suspend fun getTrackByIdSync(id: Long): TrackEntity?
}
```

Рисунок 16 – Интерфейс TracksDao

3.2 Тестирование программного продукта

Тестирование мобильного приложения Playlist Maker проводилось с целью проверки корректности работы основных функциональных возможностей, выявления возможных ошибок и оценки стабильности приложения при пользовательском взаимодействии. В процессе тестирования особое внимание

уделялось корректности бизнес-логики, работе с локальным хранилищем данных и взаимодействию с внешним API. На рисунке 17 приведен главный экран приложения.

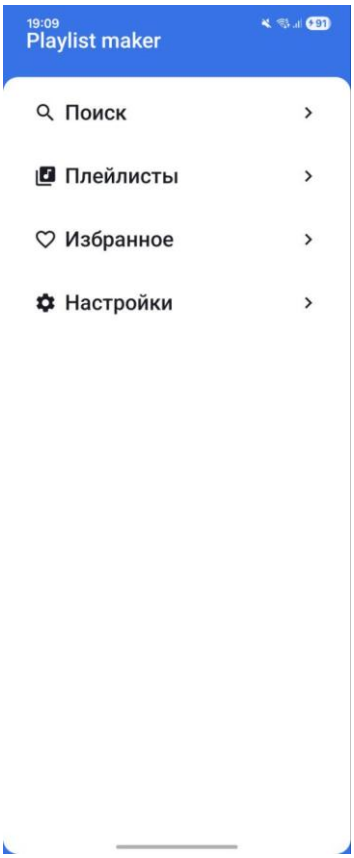

Рисунок 17 – Главный экран приложения

Была проверена работа функции поиска музыкальных композиций с использованием iTunes Search API. В ходе тестирования выполнялся ввод различных поисковых запросов, включая корректные, пустые и некорректные значения. Проверялась корректность отображения результатов поиска, обработка ситуаций отсутствия результатов, а также реакция приложения на отсутствие интернет-соединения. Также была проверена работа истории поиск (см. рис. 18).

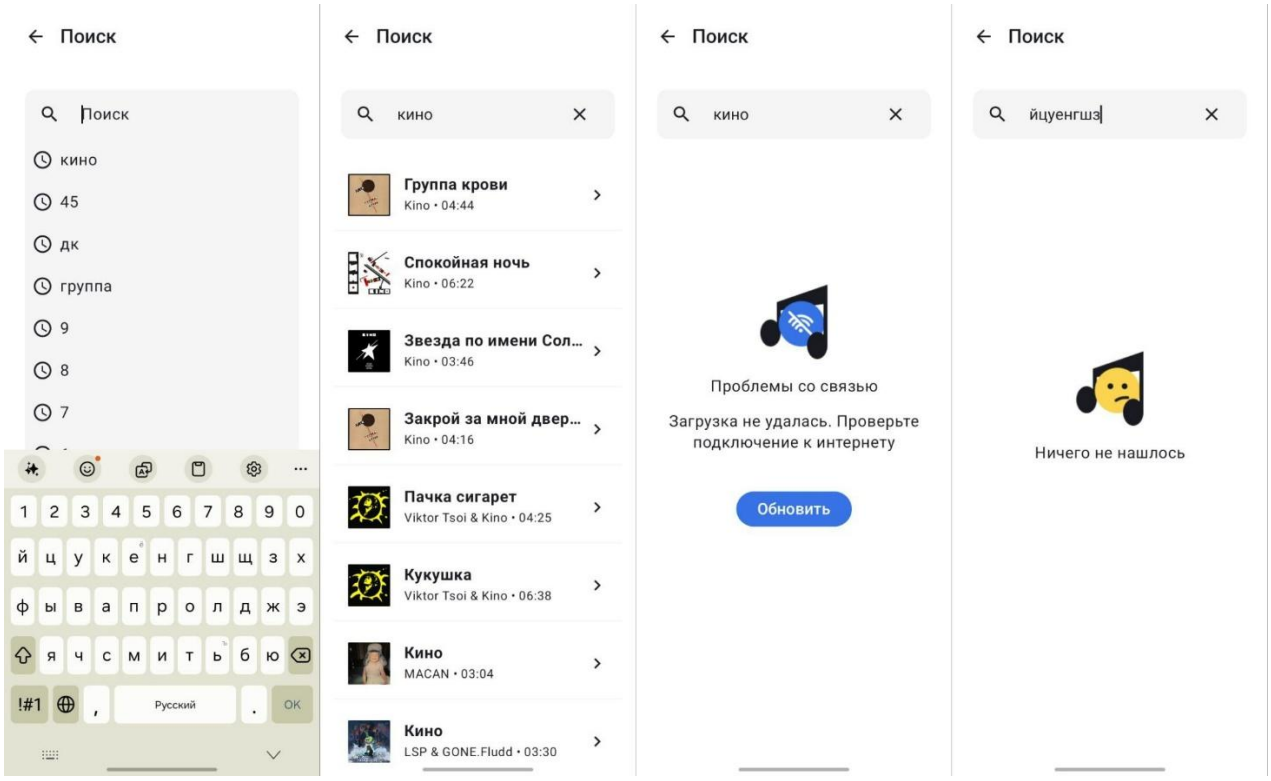

Рисунок 18 – Экран поиска

В рамках тестирования функциональности плейлистов были проверены следующие сценарии (см. рис. 19-20):

- создание нового плейлиста;
- редактирование названия и описания плейлиста;
- добавление треков в плейлист;
- удаление треков из плейлиста;
- корректное отображение количества треков и информации о плейлисте.

Особое внимание уделялось сохранению данных между перезапусками приложения, что подтверждает корректную работу локального хранилища.

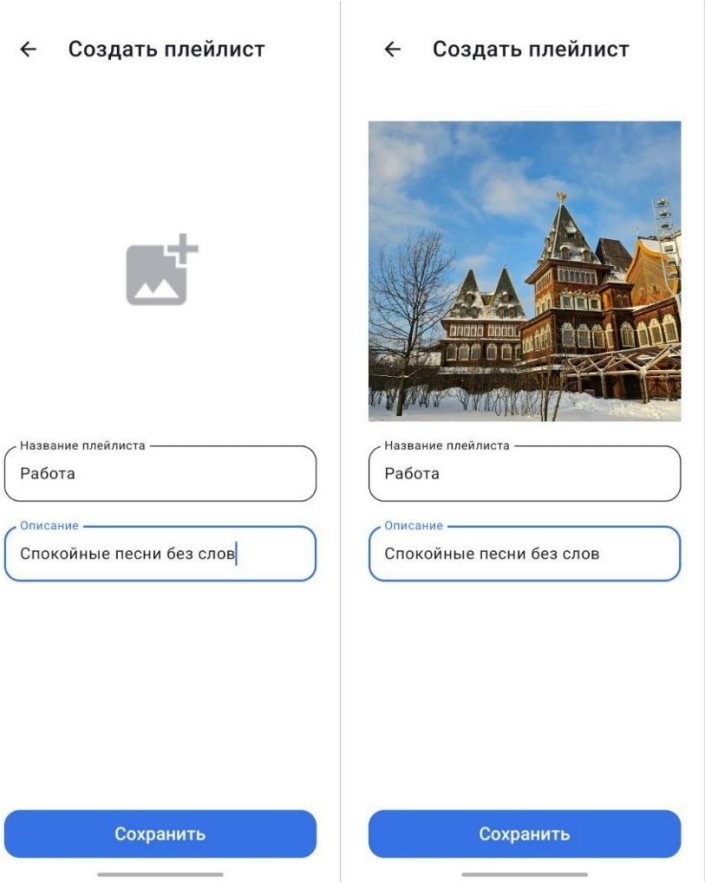

Рисунок 19 – Создание плейлиста

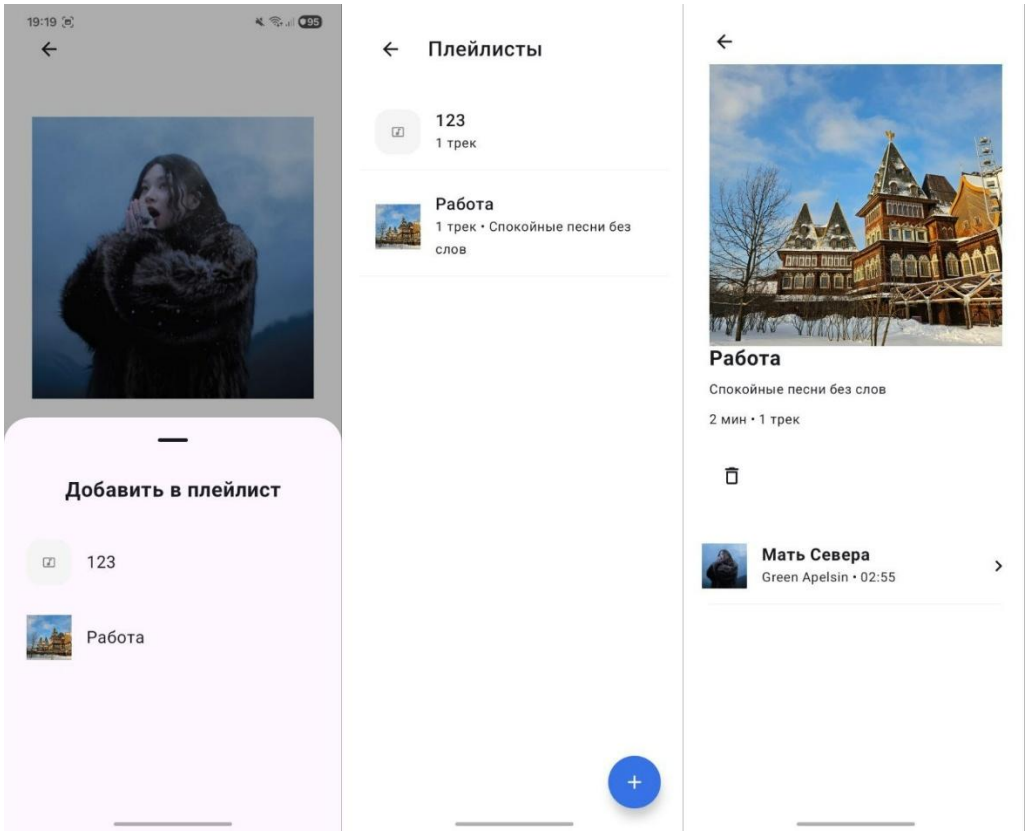

Рисунок 20 – Добавление трека в плейлист

Проверялась работа механизма добавления и удаления треков из списка избранных (см. рис. 21-22). Было подтверждено, что выбранные пользователем треки корректно сохраняются в базе данных и отображаются в соответствующем разделе приложения. Также тестировалась корректность обновления состояния избранного при переходе между экранами.

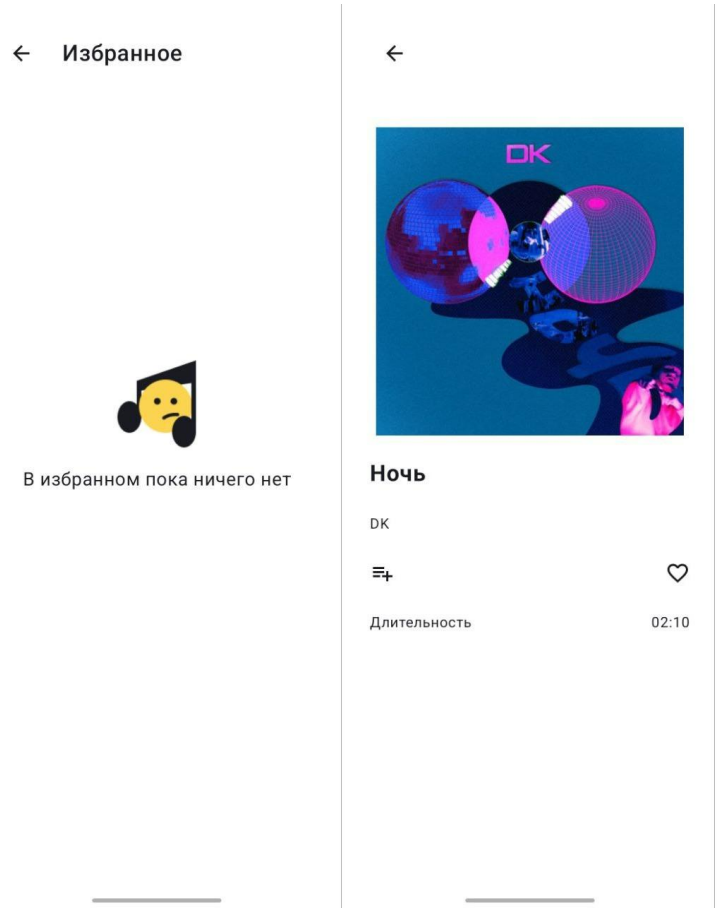

Рисунок 21 – Пустое избранное

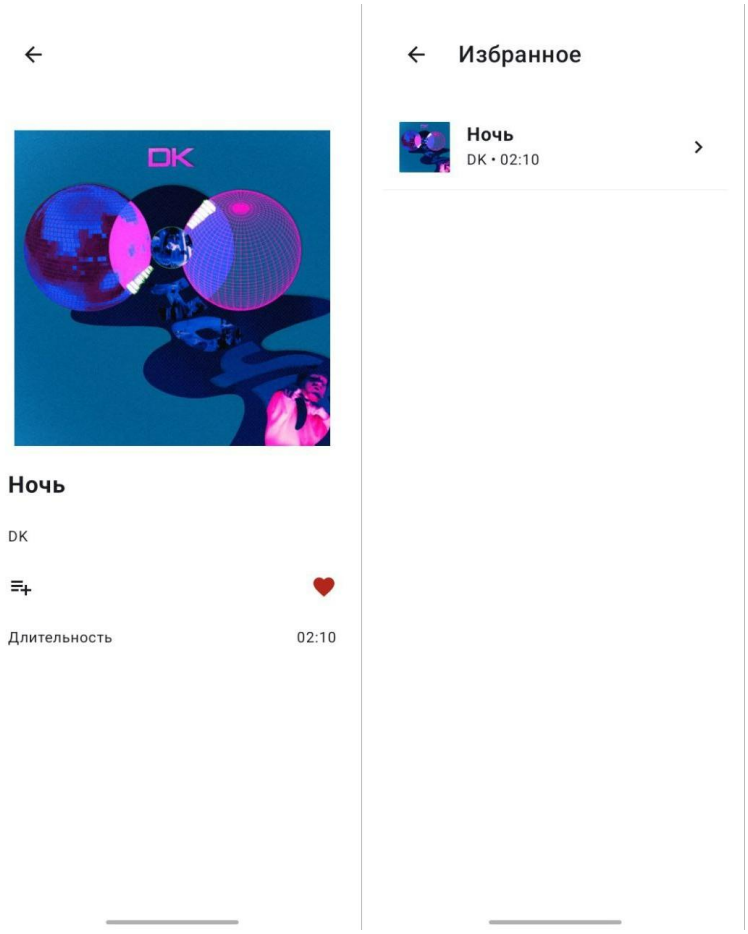

Рисунок 22 – Добавление трека в избранное

В ходе тестирования экрана настроек была проверена корректность работы элементов управления и сохранение выбранных пользователем параметров (см. рис. 23). Было проверено, что выбранная пользователем тема сохраняется при выходе из приложения, а также верно отображается на всех экранах.

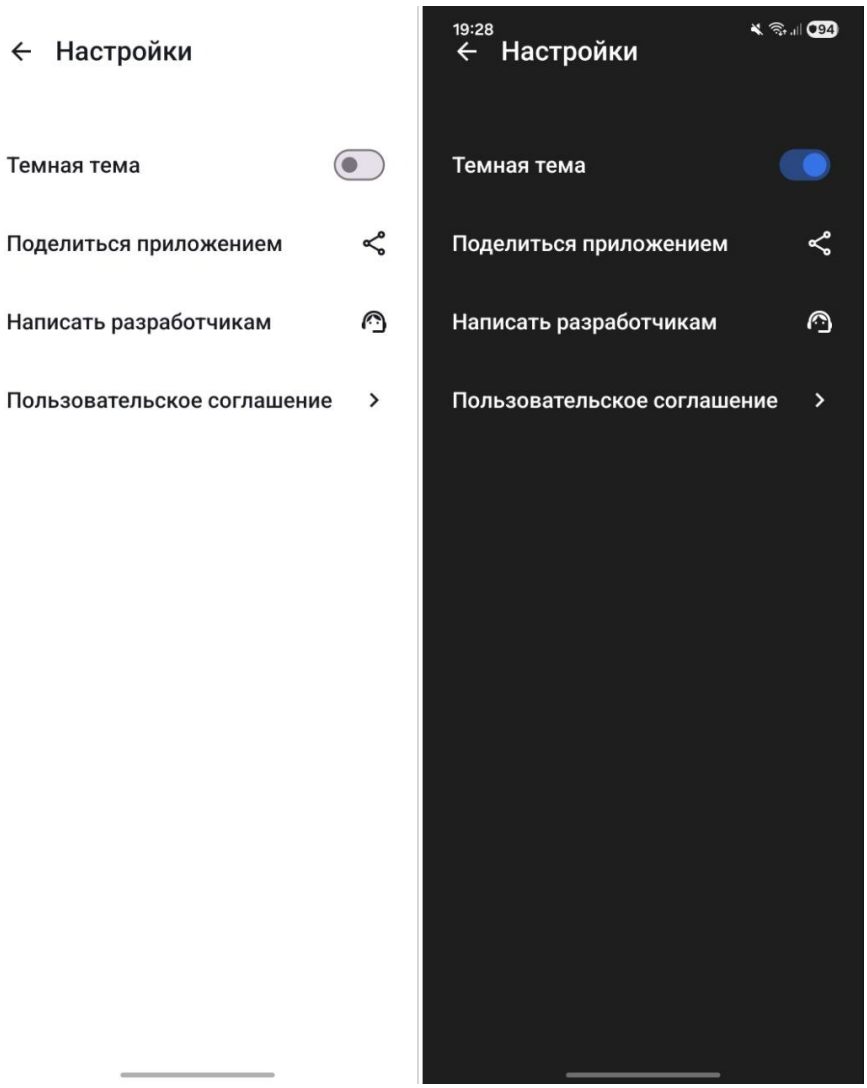

Рисунок 23 – Экран настроек

3.2 Полный листинг

Листинг 1 – Creator

```
package com.practicum.myapplication.creator

import android.content.Context
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.practicum.myapplication.data.PlaylistsRepositoryImpl
import com.practicum.myapplication.data.SearchHistoryRepositoryImpl
import com.practicum.myapplication.data.db.AppDatabase
import com.practicum.myapplication.data.db.ImageFileManager
import com.practicum.myapplication.data.network.RetrofitNetworkClient
import com.practicum.myapplication.data.network.TracksRepositoryImpl
import com.practicum.myapplication.data.preferences.SearchHistoryPreferences
import com.practicum.myapplication.data.preferences.ThemePreferences
import com.practicum.myapplication.domain.ITunesApiService
import com.practicum.myapplication.domain.PlaylistsRepository
import com.practicum.myapplication.domain.SearchHistoryRepository
import com.practicum.myapplication.domain.TracksRepository
import com.practicum.myapplication.ui.playlists.CreatePlaylistViewModel
import com.practicum.myapplication.ui.theme.ThemeViewModel
import kotlinx.coroutines.CoroutineScope
```

```
import kotlinx.coroutines.Dispatchers
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object Creator {
    private val scope = CoroutineScope(Dispatchers.IO)
    private const val BASE_URL = "https://itunes.apple.com/"

    // Создаем экземпляр Retrofit
    private val retrofit: Retrofit = Retrofit.Builder()
        .baseUrl(BASE_URL)
        .addConverterFactory(GsonConverterFactory.create())
        .build()

    // Создаем API сервис
    private val iTunesApiService: ITunesApiService =
retrofit.create(ITunesApiService::class.java)
    private var appContext: Context? = null
    fun init(context: Context) {
        appContext = context.applicationContext
    }

    fun getAppContext(): Context {
        return appContext ?: throw IllegalStateException("AppContext not
initialized")
    }

    // Метод для репозитория треков
    fun getTracksRepository(): TracksRepository {
        val context = appContext ?: throw IllegalStateException("AppContext
not initialized. Call Creator.init() first!")
        val database = AppDatabase.getInstance(context)
        return TracksRepositoryImpl(
            networkClient = RetrofitNetworkClient(iTunesApiService),
            tracksDao = database.tracksDao(),
            playlistsDao = database.playlistsDao(),
            scope = scope
        )
    }

    // Метод для репозитория плейлистов
    fun getPlaylistsRepository(): PlaylistsRepository {
        val context = appContext ?: throw IllegalStateException("AppContext
not initialized. Call Creator.init() first!")
        val database = AppDatabase.getInstance(context)
        return PlaylistsRepositoryImpl(
            playlistsDao = database.playlistsDao(),
            tracksDao = database.tracksDao(),
            scope = scope
        )
    }

    fun getTracksRepository(context: Context): TracksRepository {
        val database = AppDatabase.getInstance(context)
        return TracksRepositoryImpl(
            networkClient = RetrofitNetworkClient(iTunesApiService),
            tracksDao = database.tracksDao(),
            playlistsDao = database.playlistsDao(),
            scope = scope
        )
    }

    fun getPlaylistsRepository(context: Context): PlaylistsRepository {
```

```
        val database = AppDatabase.getInstance(context)
        return PlaylistsRepositoryImpl(
            playlistsDao = database.playlistsDao(),
            tracksDao = database.tracksDao(),
            scope = scope
        )
    }

    fun getSearchHistoryRepository(context: Context): SearchHistoryRepository
{
        val preferences = SearchHistoryPreferences(context)
        return SearchHistoryRepositoryImpl(preferences, scope)
    }

    fun getSearchHistoryRepository(): SearchHistoryRepository {
        val context = appContext ?: throw IllegalStateException("Call
Creator.init() first!")
        val preferences = SearchHistoryPreferences(context)
        return SearchHistoryRepositoryImpl(preferences, scope)
    }

    fun getNewPlaylistViewModelFactory(): ViewModelProvider.Factory =
        object : ViewModelProvider.Factory {
            @Suppress("UNCHECKED_CAST")
            override fun <T : ViewModel> create(modelClass: Class<T>): T {
                return CreatePlaylistViewModel(
                    getPlaylistsRepository(),
                    getImageFileManager()
                ) as T
            }
        }

    fun getThemeViewModelFactory(): ViewModelProvider.Factory =
        object : ViewModelProvider.Factory {
            @Suppress("UNCHECKED_CAST")
            override fun <T : ViewModel> create(modelClass: Class<T>): T {
                val themePreferences = ThemePreferences(getAppContext())
                return ThemeViewModel(themePreferences) as T
            }
        }

    fun getImageFileManager(context: Context): ImageFileManager {
        return ImageFileManager(context)
    }

    fun getImageFileManager(): ImageFileManager {
        val context = appContext ?: throw IllegalStateException("AppContext
not initialized")
        return ImageFileManager(context)
    }
}
```

Листинг 2 – PlaylistConverter

```
package com.practicum.myapplication.data.converter

import com.practicum.myapplication.data.database.entity.PlaylistEntity
import com.practicum.myapplication.domain.models.Playlist
import com.practicum.myapplication.domain.models.Track

fun PlaylistEntity.toDomain(tracks: List<Track> = emptyList()): Playlist {
    return Playlist(
        id = this.id,
        name = this.name,
        description = this.description,
        coverImageUri = this.coverImageUri,
        tracks = tracks
    )
}

fun Playlist.toEntity(): PlaylistEntity {
    return PlaylistEntity(
        id = this.id,
        name = this.name,
        description = this.description,
        coverImageUri = this.coverImageUri
    )
}
```

Листинг 3 – TrackConverter

```
package com.practicum.myapplication.data.converter

import com.practicum.myapplication.data.database.entity.TrackEntity
import com.practicum.myapplication.domain.models.Track

fun TrackEntity.toTrack(playlistIds: List<Long> = emptyList()): Track {
    return Track(
        id = this.id,
        trackName = this.trackName,
        artistName = this.artistName,
        trackTime = this.trackTime,
        image = this.image,
        favorite = this.favorite,
        playlistId = playlistIds
    )
}

fun Track.toEntity(): TrackEntity {
    return TrackEntity(
        id = this.id,
        trackName = this.trackName,
        artistName = this.artistName,
        trackTime = this.trackTime,
        image = this.image,
        favorite = this.favorite
    )
}
```

Листинг 4 – AppDatabase

```
package com.practicum.myapplication.data.db

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import com.practicum.myapplication.data.database.dao.PlaylistsDao
import com.practicum.myapplication.data.database.dao.TracksDao
import com.practicum.myapplication.data.database.entity.PlaylistEntity
import com.practicum.myapplication.data.database.entity.PlaylistTrackCrossRef
import com.practicum.myapplication.data.database.entity.TrackEntity

@Database(
    entities = [
        TrackEntity::class,
        PlaylistEntity::class,
        PlaylistTrackCrossRef::class
    ],
    version = 2,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun tracksDao(): TracksDao
    abstract fun playlistsDao(): PlaylistsDao

    companion object {
        @Volatile
        private var instance: AppDatabase? = null

        fun getInstance(context: Context): AppDatabase {
            return instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "playlist_maker.db"
                )
                    .fallbackToDestructiveMigration()
                    .build().also { instance = it }
            }
        }
    }
}
```

Листинг 5 – ImageFileManager

```
package com.practicum.myapplication.data.db

import android.content.Context
import android.net.Uri
import java.io.File
import java.io.FileOutputStream
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class ImageFileManager(private val context: Context) {

    private val imagesDir: File by lazy {
        File(context.filesDir, "playlist_covers").apply {
            if (!exists()) mkdirs()
        }
    }
    suspend fun copyImageToAppStorage(uri: Uri): Uri? {
        return try {
            val timeStamp = SimpleDateFormat("yyyyMMdd_HHmmss",
Locale.getDefault()).format(Date())
            val fileName = "playlist_cover_$timeStamp.jpg"
            val destinationFile = File(imagesDir, fileName)

            context.contentResolver.openInputStream(uri)?.use { inputStream ->
                FileOutputStream(destinationFile).use { outputStream ->
                    inputStream.copyTo(outputStream)
                }
            }
            Uri.fromFile(destinationFile)
        } catch (e: Exception) {
            e.printStackTrace()
            null
        }
    }
    fun getUriFromPath(path: String?): Uri? {
        if (path.isNullOrEmpty()) return null
        return try {
            val file = File(path)
            if (file.exists()) Uri.fromFile(file) else null
        } catch (e: Exception) {
            null
        }
    }

}
```

Листинг 6 – BaseResponse

```
package com.practicum.myapplication.data.dto

open class BaseResponse() {
    var resultCode: Int = 0
    var errorMessage: String? = null
}
```

Листинг 7 – TrackDto

```
package com.practicum.myapplication.data.dto

import com.google.gson.annotations.SerializedName

data class TrackDto(
    @SerializedName("trackId")
    val id: Long,

    @SerializedName("trackName")
    val trackName: String,

    @SerializedName("artistName")
    val artistName: String,

    @SerializedName("trackTimeMillis")
    val trackTimeMillis: Long,

    @SerializedName("previewUrl")
    val previewUrl: String?,

    @SerializedName("artworkUrl100")
    val image: String?
)
```

Листинг 8 – TracksSearchRequest

```
package com.practicum.myapplication.data.dto

data class TracksSearchRequest(val expression: String)
```

Листинг 9 – TracksSearchResponse

```
package com.practicum.myapplication.data.dto

class TracksSearchResponse(
    val resultCount: Int,
    val results: List<TrackDto>
): BaseResponse()
```

Листинг 10 – RetrofitNetworkClient

```
package com.practicum.myapplication.data.network

import com.practicum.myapplication.data.dto.TracksSearchRequest
import com.practicum.myapplication.domain.NetworkClient
import com.practicum.myapplication.domain.ITunesApiService
import com.practicum.myapplication.data.dto.BaseResponse
import java.io.IOException

class RetrofitNetworkClient(private val api: ITunesApiService) : NetworkClient
{

    override suspend fun doRequest(dto: Any): BaseResponse {
        return when (dto) {
            is TracksSearchRequest -> {
                try {
                    val response = api.searchTracks(
                        query = dto.expression,
                        media = "music",
                        entity = "song",
                        limit = 10
                    )
                    response.resultCode = 200
```

```
                    response
                } catch (e: IOException) {
                    throw IOException("Network error: ${e.message ?: "No
internet connection"}")
                } catch (e: Exception) {
                    throw Exception("Unexpected error: ${e.message}")
                }
            }
            else -> throw IllegalArgumentException("Invalid request type")
        }
    }
}
```

Листинг 11 – TracksRepositoryImpl

```
package com.practicum.myapplication.data.network

import com.practicum.myapplication.data.converter.toTrack
import com.practicum.myapplication.data.converter.toEntity
import com.practicum.myapplication.data.database.dao.PlaylistsDao
import com.practicum.myapplication.data.database.dao.TracksDao
import com.practicum.myapplication.data.database.entity.PlaylistTrackCrossRef
import com.practicum.myapplication.data.dto.TracksSearchRequest
import com.practicum.myapplication.data.dto.TracksSearchResponse
import com.practicum.myapplication.domain.NetworkClient
import com.practicum.myapplication.domain.TracksRepository
import com.practicum.myapplication.domain.models.Track
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import java.text.SimpleDateFormat
import java.util.Locale

class TracksRepositoryImpl(
    private val networkClient: NetworkClient,
    private val tracksDao: TracksDao,
    private val playlistsDao: PlaylistsDao,
    private val scope: CoroutineScope
) : TracksRepository {

    private val timeFormatter = SimpleDateFormat("mm:ss", Locale.getDefault())

    override suspend fun searchTracks(expression: String): List<Track> {
        val response =
networkClient.doRequest(TracksSearchRequest(expression))
        delay(1000)

        return if (response.resultCode == 200) {
            (response as TracksSearchResponse).results.map { it ->
                val trackTime = try {
                    timeFormatter.format(it.trackTimeMillis)
                } catch (_: Exception) {
                    "00:00"
                }

                // Проверяем, есть ли трек уже в базе
                val existingTrack = tracksDao.getTrackByIdSync(it.id)
                val playlistIds = if (existingTrack != null) {
                    playlistsDao.getPlaylistIdsForTrack(it.id)
                } else {
                    emptyList()
                }
```

```
                val track = Track(
                    id = it.id,
                    trackName = it.trackName,
                    artistName = it.artistName,
                    trackTime = trackTime,
                    image = it.image ?: "",
                    favorite = existingTrack?.favorite ?: false,
                    playlistId = playlistIds
                )

                // Сохранение трека в базу
                tracksDao.insertTrack(track.toEntity())

                track
            }
        } else {
            emptyList()
        }
    }

    override fun getTrackByNameAndArtist(track: Track): Flow<Track?> {
        return tracksDao.getTrackByNameAndArtist(track.trackName,
track.artistName)
            .map { entity ->
                entity?.let {
                    // Получаем плейлисты для этого трека
                    val playlistIds =
playlistsDao.getPlaylistIdsForTrack(it.id)
                    it.toTrack(playlistIds)
                }
            }
    }

    override fun getFavoriteTracks(): Flow<List<Track>> {
        return tracksDao.getFavoriteTracks()
            .map { entities ->
                entities.map { entity ->
                    val playlistIds =
playlistsDao.getPlaylistIdsForTrack(entity.id)
                    entity.toTrack(playlistIds)
                }
            }
    }

    override suspend fun insertSongToPlaylist(track: Track, playlistId: Long)
{
        // Сохраняем трек в базу
        tracksDao.insertTrack(track.toEntity())

        // Создаем связь трек-плейлист
        playlistsDao.insertCrossRef(
            PlaylistTrackCrossRef(
                playlistId = playlistId,
                trackId = track.id
            )
        )
    }

    override suspend fun deleteSongFromPlaylist(track: Track, playlistId:
Long) {
        playlistsDao.deleteCrossRef(playlistId, track.id)
    }
```

```
    override suspend fun updateTrackFavoriteStatus(track: Track, isFavorite:
Boolean) {
        val entity = track.toEntity().copy(favorite = isFavorite)
        tracksDao.insertTrack(entity)
    }

    override suspend fun deleteTracksByPlaylistId(playlistId: Long) {
        playlistsDao.deleteAllCrossRefsForPlaylist(playlistId)
    }

    override fun getTrackById(id: Long): Flow<Track?> {
        return tracksDao.getTrackById(id)
            .map { entity ->
                entity?.let {
                    val playlistIds =
playlistsDao.getPlaylistIdsForTrack(it.id)
                    it.toTrack(playlistIds)
                }
            }
    }
}
```

Листинг 12 – SearchHistoryPreferences

```
package com.practicum.myapplication.data.preferences

import android.content.Context
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.core.stringPreferencesKey
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

private val Context.dataStore: DataStore<Preferences> by
preferencesDataStore(name = "search_history")

private const val MAX_ENTRIES = 10
private const val SEPARATOR = ","
private val SEARCH_HISTORY_KEY = stringPreferencesKey("search_history")

class SearchHistoryPreferences(
    private val context: Context,
    private val coroutineScope: CoroutineScope =
CoroutineScope(Dispatchers.IO)
) {

    fun addEntry(word: String) {
        if (word.isEmpty()) {
            return
        }

        coroutineScope.launch {
            context.dataStore.edit { preferences ->
                val historyString = preferences[SEARCH_HISTORY_KEY].orEmpty()
                val history = if (historyString.isNotEmpty()) {
```

```
                    historyString.split(SEPARATOR).toMutableList()
                } else {
                    mutableListOf()
                }

                history.remove(word)
                history.add(0, word)

                val subList = if (history.size > MAX_ENTRIES) {
                    history.subList(0, MAX_ENTRIES) // Храним не более 10
элементов
                } else {
                    history
                }

                val updatedString = subList.joinToString(SEPARATOR)
                preferences[SEARCH_HISTORY_KEY] = updatedString
            }
        }
    }

    suspend fun getEntries(): List<String> = withContext(Dispatchers.IO) {
        getEntriesFlow().first()
    }

    fun getEntriesFlow(): Flow<List<String>> {
        return context.dataStore.data.map { preferences ->
            val historyString = preferences[SEARCH_HISTORY_KEY].orEmpty()
            if (historyString.isNotEmpty()) {
                historyString.split(SEPARATOR)
            } else {
                emptyList()
            }
        }
    }

    suspend fun clearHistory() {
        context.dataStore.edit { preferences ->
            preferences.remove(SEARCH_HISTORY_KEY)
        }
    }
}
```

Листинг 13 – ThemePreferences

```
package com.practicum.myapplication.data.preferences

import android.content.Context
import androidx.datastore.core.DataStore
import androidx.datastore.preferences.core.Preferences
import androidx.datastore.preferences.core.booleanPreferencesKey
import androidx.datastore.preferences.core.edit
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map

val Context.themeDataStore: DataStore<Preferences> by
preferencesDataStore(name = "theme_settings")

class ThemePreferences(private val context: Context) {

    companion object {
        val IS_DARK_THEME = booleanPreferencesKey("is_dark_theme")
    }

    suspend fun setDarkTheme(isDarkTheme: Boolean) {
        context.themeDataStore.edit { preferences ->
            preferences[IS_DARK_THEME] = isDarkTheme
        }
    }

    val isDarkTheme: Flow<Boolean> = context.themeDataStore.data
        .map { preferences ->
            preferences[IS_DARK_THEME] ?: false
        }
}
```

Листинг 14 – PlaylistsRepositoryImpl

```
package com.practicum.myapplication.data

import com.practicum.myapplication.data.converter.toDomain
import com.practicum.myapplication.data.converter.toTrack
import com.practicum.myapplication.data.database.dao.PlaylistsDao
import com.practicum.myapplication.data.database.dao.TracksDao
import com.practicum.myapplication.domain.PlaylistsRepository
import com.practicum.myapplication.domain.models.Playlist
import com.practicum.myapplication.domain.models.Track
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.combine

class PlaylistsRepositoryImpl(
    private val playlistsDao: PlaylistsDao,
    private val tracksDao: TracksDao,
    private val scope: CoroutineScope
) : PlaylistsRepository {

    override fun getPlaylist(playlistId: Long): Flow<Playlist?> {
        return playlistsDao.getPlaylistById(playlistId)
            .combine(tracksDao.getAllTracksFlow()) { playlistEntity,
allTrackEntities  ->
                playlistEntity?.let { entity ->
                    // Получаем треки для этого плейлиста
                    val trackIds =
playlistsDao.getTrackIdsForPlaylist(playlistId)
                    val tracks = allTrackEntities
```

```
                        .filter { trackIds.contains(it.id) }
                        .map { trackEntity ->
                            val playlistIds =
playlistsDao.getPlaylistIdsForTrack(trackEntity.id)
                            trackEntity.toTrack(playlistIds)
                        }

                    entity.toDomain(tracks)
                }
            }
    }

    override fun getAllPlaylists(): Flow<List<Playlist>> {
        return playlistsDao.getAllPlaylists()
            .combine(tracksDao.getAllTracksFlow()) { playlistEntities,
allTrackEntities  ->
                playlistEntities.map { playlistEntity ->
                    // Получаем треки для каждого плейлиста
                    val trackIds =
playlistsDao.getTrackIdsForPlaylist(playlistEntity.id)
                    val tracks = allTrackEntities
                        .filter { trackIds.contains(it.id) }
                        .map { trackEntity ->
                            val playlistIds =
playlistsDao.getPlaylistIdsForTrack(trackEntity.id)
                            trackEntity.toTrack(playlistIds)
                        }

                    playlistEntity.toDomain(tracks)
                }
            }
    }

    override suspend fun addNewPlaylist(name: String, description: String,
coverImageUri: String?) {
        val entity =
com.practicum.myapplication.data.database.entity.PlaylistEntity(
            name = name,
            description = description,
            coverImageUri = coverImageUri
        )
        playlistsDao.insertPlaylist(entity)
    }

    override suspend fun deletePlaylistById(id: Long) {
        // Удаляем связи трек-плейлист
        playlistsDao.deleteAllCrossRefsForPlaylist(id)
        // Удаляем сам плейлист
        playlistsDao.deletePlaylistById(id)
    }
}
```

Листинг 15 – SearchHistoryImpl

```
package com.practicum.myapplication.data

import com.practicum.myapplication.data.preferences.SearchHistoryPreferences
import com.practicum.myapplication.domain.SearchHistoryRepository
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow

class SearchHistoryRepositoryImpl(
    private val preferences: SearchHistoryPreferences,
    private val scope: CoroutineScope = CoroutineScope(Dispatchers.IO)
) : SearchHistoryRepository {

    override fun addToHistory(query: String) {
        preferences.addEntry(query)
    }

    override suspend fun getHistory(): List<String> {
        return preferences.getEntries()
    }

    override fun getHistoryFlow(): Flow<List<String>> {
        return preferences.getEntriesFlow()
    }

    override suspend fun clearHistory() {
        preferences.clearHistory()
    }
}
```

Листинг 16 – ITunesApiService

```
package com.practicum.myapplication.domain

import com.practicum.myapplication.data.dto.TracksSearchResponse
import retrofit2.http.GET
import retrofit2.http.Query

interface ITunesApiService {
    @GET("search")
    suspend fun searchTracks(
        @Query("term") query: String,
        @Query("media") media: String = "music",
        @Query("entity") entity: String = "song",
        @Query("limit") limit: Int = 50
    ): TracksSearchResponse
}
```

Листинг 17 – AppActivity

```
package com.practicum.myapplication.ui.activity

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.ui.Modifier
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.collectAsState
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.compose.rememberNavController
import com.practicum.myapplication.ui.navigation.PlaylistHost
```

```
import com.practicum.myapplication.creator.Creator
import com.practicum.myapplication.ui.theme.PlaylistMakerTheme
import com.practicum.myapplication.ui.theme.ThemeViewModel

class AppActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Creator.init(this)
        setContent {
            val themeViewModel: ThemeViewModel = viewModel(
                factory = Creator.getThemeViewModelFactory()
            )
            val isDarkTheme =
themeViewModel.isDarkTheme.collectAsState().value

            PlaylistMakerTheme(darkTheme = isDarkTheme) {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    val navController = rememberNavController()
                    PlaylistHost(navController = navController)
                }
            }
        }
    }
}
```

Листинг 18 – Navigation

```
package com.practicum.myapplication.ui.navigation

// Enum класс для перечисления экранов
enum class Screen {
    MAIN,
    SEARCH,
    SETTINGS,
    PLAYLISTS,
    FAVORITES,
    TRACK_DETAILS,
    CREATE_PLAYLIST,
    PLAYLIST_SCREEN
}
```

Листинг 19 – PlaylistHost

```
package com.practicum.myapplication.ui.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.navArgument
import com.practicum.myapplication.ui.main.MainScreen
import com.practicum.myapplication.ui.search.SearchScreen
import com.practicum.myapplication.ui.settings.SettingsScreen
import com.practicum.myapplication.ui.favorites.FavoritesScreen
import com.practicum.myapplication.ui.playlists.PlaylistsScreen
import com.practicum.myapplication.ui.trackdetails.TrackDetailsScreen
import com.practicum.myapplication.ui.playlists.CreatePlaylistScreen
import com.practicum.myapplication.ui.playlists.PlaylistDetailViewModel
import com.practicum.myapplication.ui.playlists.PlaylistScreen
```

```
@Composable
fun PlaylistHost(
    navController: NavHostController
) {
    NavHost(
        navController = navController,
        startDestination = Screen.MAIN.name
    ) {
        // Главный экран
        composable(Screen.MAIN.name) {
            MainScreen(
                onSearchClick = {
                    navController.navigate(Screen.SEARCH.name)
                },
                onSettingsClick = {
                    navController.navigate(Screen.SETTINGS.name)
                },
                onPlaylistsClick = {
                    navController.navigate(Screen.PLAYLISTS.name)
                },
                onFavoritesClick = {
                    navController.navigate(Screen.FAVORITES.name)
                }
            )
        }

        // Экран поиска
        composable(Screen.SEARCH.name) {
            SearchScreen(
                onBackClick = {
                    navController.popBackStack()
                },
                onTrackClick = { trackId ->

navController.navigate("${Screen.TRACK_DETAILS.name}/${trackId}")
                }
            )
        }

        // Экран настроек
        composable(Screen.SETTINGS.name) {
            SettingsScreen(
                onBackClick = {
                    navController.popBackStack()
                }
            )
        }

        // Экран плейлистов
        composable(Screen.PLAYLISTS.name) {
            PlaylistsScreen(
                onBackClick = {
                    navController.popBackStack()
                },
                onCreatePlaylistClick = {
                    navController.navigate(Screen.CREATE_PLAYLIST.name)
                },
                onPlaylistClick = { playlistId ->

navController.navigate("${Screen.PLAYLIST_SCREEN.name}/$playlistId")
                }
            )
        }
```

```
        // Экран плейлиста
        composable(
            route = "${Screen.PLAYLIST_SCREEN.name}/{playlistId}",
            arguments = listOf(
                navArgument("playlistId") {
                    type = NavType.LongType
                }
            )
        ) { backStackEntry ->
            val playlistId = backStackEntry.arguments?.getLong("playlistId")
?: 0L

            PlaylistScreen(
                playlistId = playlistId,
                onBackClick = {
                    navController.popBackStack()
                },
                onTrackClick = { trackId ->

navController.navigate("${Screen.TRACK_DETAILS.name}/${trackId}")
                },
                onPlaylistDeleted = {
                    navController.popBackStack()
                },
                viewModelFactory =
PlaylistDetailViewModel.getViewModelFactory(playlistId)
            )
        }

        // Экран избранного
        composable(Screen.FAVORITES.name) {
            FavoritesScreen(
                onBackClick = {
                    navController.popBackStack()
                },
                onTrackClick = { trackId  ->

navController.navigate("${Screen.TRACK_DETAILS.name}/$trackId")
                }
            )
        }

        // Экран деталей трека
        composable(
            route = "${Screen.TRACK_DETAILS.name}/{trackId}",
            arguments = listOf(
                navArgument("trackId") { type = NavType.LongType }
            )
        ) { backStackEntry ->
            val trackId = backStackEntry.arguments?.getLong("trackId") ?: 0L

            // Получаем трек из репозитория
            TrackDetailsScreen(
                trackId = trackId,
                onBackClick = { navController.popBackStack() }
            )
        }

        // Экран создания плейлиста
        composable(Screen.CREATE_PLAYLIST.name) {
            CreatePlaylistScreen(
                onBackClick = {
```

```
                    navController.popBackStack()
                },
                viewModel = androidx.lifecycle.viewmodel.compose.viewModel(
                    factory =
com.practicum.myapplication.ui.playlists.CreatePlaylistViewModel.getViewModelF
actory())
            )
        }
    }
}
```

Листинг 20 – CreatePlaylistViewModel

```
package com.practicum.myapplication.ui.playlists

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import com.practicum.myapplication.data.db.ImageFileManager
import com.practicum.myapplication.domain.PlaylistsRepository
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class CreatePlaylistViewModel(
    private val playlistsRepository: PlaylistsRepository,
    private val imageFileManager: ImageFileManager
) : ViewModel() {

    // Храним временный Uri (из галереи)
    private var tempImageUri: String? = null

    // Flow для превью в UI
    private val _coverImageUri = MutableStateFlow<String?>(null)
    val coverImageUri = _coverImageUri.asStateFlow()

    // Устанавливаем временный Uri при выборе картинки
    fun setTempImageUri(uri: String) {
        tempImageUri = uri
        _coverImageUri.value = uri  // Показываем превью
    }

    // Создаем плейлист с копированием картинки
    fun createNewPlaylist(name: String, description: String) {
        viewModelScope.launch {
            var finalImageUri: String? = null

            // Если есть выбранная картинка - копируем её
            tempImageUri?.let { uriString ->
                // Преобразуем строку в Uri
                val uri = android.net.Uri.parse(uriString)
                val copiedUri = imageFileManager.copyImageToAppStorage(uri)
                finalImageUri = copiedUri?.toString()
            }

            // Создаем плейлист с сохраненным Uri
            playlistsRepository.addNewPlaylist(
                name = name,
                description = description,
                coverImageUri = finalImageUri  // Используем сохраненный Uri
            )

            // Сбрасываем временные данные
```

```
            tempImageUri = null
            _coverImageUri.value = null
        }
    }

    // Метод для получения Uri из сохраненного пути (для отображения)
    fun getImageUriFromSavedPath(path: String?): String? {
        if (path.isNullOrEmpty()) return null
        val uri = imageFileManager.getUriFromPath(path)
        return uri?.toString()
    }

    companion object {
        fun getViewModelFactory(): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    val repository =
com.practicum.myapplication.creator.Creator.getPlaylistsRepository()
                    val imageFileManager =
com.practicum.myapplication.creator.Creator.getImageFileManager()
                    return CreatePlaylistViewModel(repository,
imageFileManager) as T
                }
            }
    }
}
```

Листинг 21 – PlaylistDetailViewModel

```
package com.practicum.myapplication.ui.playlists

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.practicum.myapplication.creator.Creator
import com.practicum.myapplication.domain.PlaylistsRepository
import androidx.lifecycle.viewModelScope
import com.practicum.myapplication.data.db.ImageFileManager
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class PlaylistDetailViewModel(
    private val playlistsRepository: PlaylistsRepository,
    private val imageFileManager: ImageFileManager,
    private val playlistId: Long
) : ViewModel() {

    val playlist = playlistsRepository.getPlaylist(playlistId)

    fun deletePlaylist() {
        viewModelScope.launch(Dispatchers.IO) {
            playlistsRepository.deletePlaylistById(playlistId)
        }
    }

    fun getImageUriForPlaylist(coverImageUri: String?): String? {
        return imageFileManager.getUriFromPath(coverImageUri)?.toString()
    }

    companion object {
        fun getViewModelFactory(playlistId: Long): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
```

```
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    return PlaylistDetailViewModel(
                        Creator.getPlaylistsRepository(),
                        Creator.getImageFileManager(),
                        playlistId
                    ) as T
                }
            }
    }
}
```

Листинг 22 – PlaylistViewModel

```
package com.practicum.myapplication.ui.playlists

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import com.practicum.myapplication.creator.Creator
import com.practicum.myapplication.data.db.ImageFileManager
import com.practicum.myapplication.domain.PlaylistsRepository
import com.practicum.myapplication.domain.TracksRepository
import com.practicum.myapplication.domain.models.Playlist
import com.practicum.myapplication.domain.models.Track
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class PlaylistViewModel(
    private val playlistsRepository: PlaylistsRepository,
    private val tracksRepository: TracksRepository,
    private val imageFileManager: ImageFileManager
) : ViewModel() {

    val playlists: Flow<List<Playlist>> =
playlistsRepository.getAllPlaylists()

    private val _favoriteTracks = MutableStateFlow<List<Track>>(emptyList())
    val favoriteTracks = _favoriteTracks.asStateFlow()

    init {
        loadFavoriteTracks()
    }

    fun createNewPlaylist(name: String, description: String) {
        viewModelScope.launch(Dispatchers.IO) {
            playlistsRepository.addNewPlaylist(name, description)
        }
    }

    fun deleteTrackFromPlaylist(track: Track, playlistId: Long) {
        viewModelScope.launch(Dispatchers.IO) {
            tracksRepository.deleteSongFromPlaylist(track, playlistId)
        }
    }

    fun getPlaylistById(id: Long): Flow<Playlist?> {
        return playlistsRepository.getPlaylist(id)
    }

    fun addTrackToPlaylist(track: Track, playlistId: Long) {
```

```
        viewModelScope.launch(Dispatchers.IO) {
            tracksRepository.insertSongToPlaylist(track, playlistId)
        }
    }

    fun toggleFavorite(track: Track, isFavorite: Boolean) {
        viewModelScope.launch(Dispatchers.IO) {
            tracksRepository.updateTrackFavoriteStatus(track, isFavorite)
            loadFavoriteTracks()
        }
    }

    fun deletePlaylist(id: Long) {
        viewModelScope.launch(Dispatchers.IO) {
            playlistsRepository.deletePlaylistById(id)
        }
    }

    fun getTrackStatus(track: Track): Flow<Track?> {
        return tracksRepository.getTrackByNameAndArtist(track)
    }

    private fun loadFavoriteTracks() {
        viewModelScope.launch(Dispatchers.IO) {
            tracksRepository.getFavoriteTracks().collect { favorites ->
                _favoriteTracks.value = favorites
            }
        }
    }

    fun getImageUriForPlaylist(coverImageUri: String?): String? {
        return imageFileManager.getUriFromPath(coverImageUri)?.toString()
    }


    companion object {
        fun getViewModelFactory(): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    return PlaylistViewModel(
                        Creator.getPlaylistsRepository(),
                        Creator.getTracksRepository(),
                        Creator.getImageFileManager()
                    ) as T
                }
            }
    }
}
```

Листинг 23 – SearchState

```
package com.practicum.myapplication.ui.search

import com.practicum.myapplication.domain.models.Track

sealed class SearchState {
    object Initial: SearchState() // Первоначальное состояние экрана
    object Searching: SearchState() // Состояние экрана при начале поиска
    data class Success(val list: List<Track>): SearchState() // Состояние
экрана при успешном завершении поиска
    data class Fail(val errorMessage: String, val lastQuery: String = ""):
SearchState() // Состояние экрана, если при запросе к серверу произошла ошибка
}
```

Листинг 24 – SearchViewModel

```
package com.practicum.myapplication.ui.search

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import com.practicum.myapplication.creator.Creator
import com.practicum.myapplication.domain.SearchHistoryRepository
import com.practicum.myapplication.domain.TracksRepository
import com.practicum.myapplication.ui.search.SearchState
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import java.io.IOException

class SearchViewModel(
    private val tracksRepository: TracksRepository,
    private val searchHistoryRepository: SearchHistoryRepository
) : ViewModel() {
    private val _searchScreenState =
MutableStateFlow<SearchState>(SearchState.Initial)
    val searchScreenState  = _searchScreenState.asStateFlow()
    private var lastQuery = ""
    private var failedQuery = ""
    val searchHistory = searchHistoryRepository.getHistoryFlow()

    fun search(whatSearch: String) {
        viewModelScope.launch(Dispatchers.IO) {
            try {
                _searchScreenState.update { SearchState.Searching }
                val list = tracksRepository.searchTracks(expression =
whatSearch)
                _searchScreenState.update { SearchState.Success(list = list) }
                // Сохраняем только при успешном поиске
                if (list.isNotEmpty()) {
                    addToHistory(whatSearch)
                }
            } catch (e: IOException) {
                failedQuery = whatSearch // Сохраняем неудавшийся запрос
                _searchScreenState.update {
                    SearchState.Fail(
                        errorMessage = e.message.toString(),
                        lastQuery = whatSearch
                    )
                }
            }
```

```
        }
    }

    fun getLastQuery(): String = lastQuery

    fun getFailedQuery(): String = failedQuery

    fun retryLastFailedSearch() {
        if (failedQuery.isNotEmpty()) {
            search(failedQuery)
        }
    }

    fun clearSearch() {
        _searchScreenState.update { SearchState.Initial }
        lastQuery = ""
    }

    fun addToHistory(query: String) {
        if (query.isNotBlank()) {
            searchHistoryRepository.addToHistory(query)
        }
    }

    suspend fun getHistoryList(): List<String> {
        return searchHistoryRepository.getHistory()
    }

    suspend fun clearHistory() {
        searchHistoryRepository.clearHistory()
    }

    companion object {
        fun getViewModelFactory(): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    return SearchViewModel(
                        Creator.getTracksRepository(),
                        Creator.getSearchHistoryRepository()
                    ) as T
                }
            }
    }
}
```

Листинг 25 – ThemeViewModel

```
package com.practicum.myapplication.ui.theme

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import com.practicum.myapplication.data.preferences.ThemePreferences
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class ThemeViewModel(
    private val themePreferences: ThemePreferences
) : ViewModel() {
```

```
    private val _isDarkTheme = MutableStateFlow(false)
    val isDarkTheme: StateFlow<Boolean> = _isDarkTheme.asStateFlow()

    init {
        loadThemePreference()
    }

    private fun loadThemePreference() {
        viewModelScope.launch {
            themePreferences.isDarkTheme.collect { isDark ->
                _isDarkTheme.value = isDark
            }
        }
    }

    fun setDarkTheme(isDark: Boolean) {
        viewModelScope.launch {
            themePreferences.setDarkTheme(isDark)
            _isDarkTheme.value = isDark
        }
    }

    companion object {
        fun getViewModelFactory(): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    val themePreferences = ThemePreferences(

com.practicum.myapplication.creator.Creator.getAppContext()
                    )
                    return ThemeViewModel(themePreferences) as T
                }
            }
    }
}
```

Листинг 26 – TrackDetailViewModel

```
package com.practicum.myapplication.ui.trackdetails

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.practicum.myapplication.creator.Creator
import com.practicum.myapplication.domain.TracksRepository
import com.practicum.myapplication.domain.models.Track
import kotlinx.coroutines.flow.Flow

class TrackDetailsViewModel(
    private val tracksRepository: TracksRepository,
    private val trackId: Long
) : ViewModel() {

    val track: Flow<Track?> = tracksRepository.getTrackById(trackId)

    companion object {
        fun getViewModelFactory(trackId: Long): ViewModelProvider.Factory =
            object : ViewModelProvider.Factory {
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T {
                    val repository = Creator.getTracksRepository()
                    return TrackDetailsViewModel(repository, trackId) as T
                }
            }
    }
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы была успешно спроектирована и разработана архитектура мобильного приложения «Playlist Maker» для операционной системы Android. Приложение предназначено для поиска музыкальных треков через внешний API, создания персонализированных плейлистов и управления пользовательской музыкальной коллекцией с возможностью локального хранения данных.

Основные достижения и реализованные задачи:

1. Проведен анализ предметной области и существующих музыкальных сервисов, на основе которого сформулированы функциональные требования и техническое задание.
2. Разработана архитектура системы с использованием методологий моделирования IDEF0 и DFD. Обоснован выбор технологического стека (Kotlin, Android Studio, Jetpack Compose, Room), обеспечивающего модульность и производительность.
3. Реализованы основные компоненты приложения, включая сетевой слой для работы с iTunes Search API, локальное хранение данных с помощью Room, систему навигации и адаптивный пользовательский интерфейс.
4. Проверена работоспособность всех функций — поиска, управления плейлистами и избранным, смены темы, а также корректность обработки сетевых ошибок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Руководство по языку Kotlin [Электронный ресурс]. – 2024. – Режим доступа: https://kotlinlang.ru/docs, свободный. – Дата обращения: 05.12.2025.
2. Основы создания интерфейса Android Studio [Электронный ресурс]. – URL: https://metanit.com/java/android/3.1.php (дата обращения: 07.12.2025).
3. Сомон П. И. Волшебство Kotlin / пер. с англ. А. Н. Киселева. — Москва: ДМК Пресс, 2020. — 536 с. — ISBN 978-5-97060-801-2. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/140599 (дата обращения: 10.12.2025). — Режим доступа: для авториз. пользователей.
4. Сохраните данные в локальной базе данных с помощью Room [Электронный ресурс]. – URL: https://developer.android.com/training/data-storage/room?hl=ru (дата обращения: 14.12.2025).
5. Льюис Ш., Данн М. Нативная разработка мобильных приложений: руководство / пер. с англ. А. Н. Киселева. — Москва: ДМК Пресс, 2020. — 376 с. — ISBN 978-5-97060-845-6. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/179491 (дата обращения: 18.12.2025). — Режим доступа: для авториз. пользователей.
6. Фрагменты [Электронный ресурс]. – URL: https://metanit.com/java/android/8.1.php (дата обращения: 21.12.2025).
7. Петросян Л. Э., Гусев К. В. Разработка мобильных приложений на языке Kotlin: учебное пособие для вузов. — Санкт-Петербург: Лань, 2024. — 104 с. — ISBN 978-5-507-49186-5. — Текст: электронный // Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/173103 (дата обращения: 24.12.2025). — Режим доступа: для авториз. пользователей.